CHAPTER I
INTRODUCTION

Programming has become an essential skill in the digital era, serving as the foundation for innovation, problem-solving, and technological advancement. However, one of the persistent challenges in programming education is the mismatch between the difficulty of coding tasks and the varying skill levels of learners. Students often encounter problems that are either too complex, leading to frustration and disengagement, or too simple, resulting in boredom and lack of motivation. Traditional teaching methods, which rely on static problem sets and manual evaluation, are limited in their ability to adapt to individual learning needs. This gap highlights the necessity of intelligent systems that can dynamically assess programming difficulty and simultaneously enhance student engagement.

In response to this challenge, I am developing a tool for learning programming that integrates machine learning techniques to evaluate coding problem difficulty and foster sustained engagement among learners. This tool is designed not only to classify programming tasks according to their complexity but also to analyze student coding behaviors, such as time spent on tasks, error frequency, and submission patterns. By combining difficulty assessment with engagement analytics, the system aims to create a more adaptive and personalized learning environment where students are challenged appropriately and motivated to persist in their learning journey.

The significance of this study lies in its potential to transform programming education through the integration of **machine learning (ML) techniques**. By

employing predictive models, the system can classify programming problems according to difficulty levels, ensuring that learners are presented with tasks appropriate to their capabilities. At the same time, the system incorporates **engagement analytics**, monitoring coding behaviors such as time spent on tasks, frequency of submissions, and error trends. These insights allow educators to identify early signs of disengagement and provide timely interventions, while learners benefit from personalized feedback that sustains motivation and improves coding performance. Such a system not only addresses the cognitive dimension of programming but also the affective aspect, fostering persistence and resilience in learners.

PROJECT CONTEXT

The purpose of this study is to design and develop an **intelligent machine learning system** that serves two primary functions: (1) to assess the difficulty of programming tasks using data-driven approaches, and (2) to enhance coding engagement through adaptive feedback and personalized recommendations. By combining these functions, the system aims to create a holistic learning environment where students are challenged at the right level and supported in maintaining active participation. For educators, the system provides a scalable tool to manage large cohorts and implement data-driven teaching strategies. For institutions, it contributes to improved learning outcomes and supports the modernization of educational practices in line with global trends in artificial intelligence and digital pedagogy.

Why do we need to assess programming difficulty among future IT enthusiast? assessing programming difficulty and actively enhancing student

engagement are critical gaps in current computer science education that directly impact learning outcomes. An intelligent machine learning system designed to evaluate the inherent difficulty of programming tasks and to tailor engagement strategies can provide personalized feedback, adaptive learning paths, and early detection of struggle, thereby reduce dropout rates and accelerate skill acquisition. By developing and validating such a system, the researchers aim to create evidence-based tools that help instructors allocate attention effectively, design better curricula, and provide students with timely, data-driven support; ultimately improving proficiency, motivation, and retention in programming courses across diverse learner populations.

Programming has become a critical skill in modern education and the IT industry. However, IT students often struggle to identify the difficulty of programming tasks and maintain engagement throughout learning. Traditional assessment methods rely heavily on human instructors and static tests, which may not capture the nuanced challenges faced by learners. This research aims to develop an intelligent system that uses machine learning to dynamically assess programming difficulty and enhance coding engagement, providing real-time feedback and tailored guidance to learners.

Traditional learning approaches, including textbooks, lectures, and static exercises, often fail to accommodate the diverse learning speeds and skill levels of students. As a result, many learners encounter difficulties in understanding programming concepts, completing tasks, and maintaining consistent engagement, which can ultimately affect their academic performance and professional readiness.

Programming education presents unique challenges that require innovative solutions. The process of learning to program is iterative and complex, as students frequently encounter errors, bugs, and unexpected outcomes that can hinder motivation and engagement. Moreover, instructors often face difficulties in providing individualized guidance to a large number of students, making it challenging to address the specific learning needs of each learner.

Currently, programming courses often rely on standardized exercises, quizzes, or manual evaluation of code submissions. These methods do not provide instant feedback on task difficulty or learner engagement, limiting instructors' ability to personalize instruction and students' ability to self-assess their skills.

Gaps or Problems of the Organization or Society (Based on Survey):

- Students are unable to gauge task difficulty accurately, resulting in frustration or disengagement.
- Instructors face challenges in monitoring engagement and progress of multiple students efficiently.
- Existing systems do not provide predictive insights or adapt to individual learning patterns.

Lack of dynamic difficulty assessment tools:

- **Technical background:** Integration of machine learning algorithms (e.g., clustering, regression, classification) for analyzing programming tasks and learner behavior.
- **Testing:** Simulated datasets of code assignments and engagement metrics will be used to evaluate system accuracy.

The drive behind this research is to improve programming education by leveraging intelligent systems that adapt to individual learner needs. The project aims to reduce disengagement, enhance learning outcomes, and provide instructors with actionable insights.

PURPOSE AND DESCRIPTION

Purpose

The purpose of “An Intelligent Machine Learning System for Programming Difficulty Assessment and Coding Engagement Enhancement” is to provide an intelligent and systematic approach to identifying students’ difficulties in programming. The system is designed to help students and instructors understand coding performance through automated assessment, error detection, prediction, recommendations, and performance reporting.

The system aims to reduce the difficulty of manually evaluating programming activities by providing automated analysis of submitted source code. It can help identify common coding errors, determine areas where students may be struggling, and provide recommendations that can guide students toward improving their programming knowledge and skills.

Description

The proposed system is a machine learning-based programming assessment platform designed primarily for students learning programming. It consists of several interconnected features that manage student information, assess coding activities, analyze programming performance, predict difficulties, and generate assessment reports.

The Student Management Module stores and manages student information and their corresponding assessment records. This provides instructors or

administrators with an organized way of monitoring individual student performance. The Coding Assessment Module allows students to submit or enter their source code for evaluation. The system identifies the programming language being used and scans the submitted code to determine whether it contains errors or produces correct results. It can analyze coding issues and provide information that may help identify the student's programming difficulties. The Prediction and Recommendation Module use the collected assessment and coding-performance data to identify patterns in student performance. Based on the assessment results, the system predicts the student's level or areas of programming difficulty and provides personalized recommendations, such as topics to review, programming concepts to practice, or coding exercises to improve specific weaknesses.

Finally, the Evaluation and Reporting Module summarizes assessment results and generates reports that can be used by students and instructors to monitor coding performance. These reports may include assessment scores, detected errors, identified areas of difficulty, predicted performance or difficulty level, recommendations, and progress over time.

Overall, the system is intended to serve as an intelligent learning support tool that combines coding assessment, machine learning-based prediction, personalized recommendations, and performance reporting to help students improve their programming abilities and become more engaged in coding activities.

OBJECTIVES

General Objective

To develop an intelligent machine learning system that assesses students' programming difficulties, analyzes their coding performance, predicts areas of

weakness, and provides personalized recommendations to enhance coding skills and engagement.

Specific Objectives

1. To develop a Student Management Module that allows the system to efficiently manage and organize student profiles, academic information, assessment records, and coding performance data.
2. To develop an Intelligent Coding Assessment Module that evaluates students' source code by identifying the programming language and analyzing the code for errors, correctness, and possible programming difficulties using intelligent code-scanning techniques.
3. To implement a Machine Learning-Based Prediction, Recommendation, and Reporting Module that analyzes assessment results to predict students' programming difficulty levels, provide appropriate learning recommendations, and generate comprehensive reports of their coding assessment performance.

SIGNIFICANCE OF THE PROJECT (SDGS)

The study is significant because it proposes an intelligent machine learning system designed to assist students and educators in addressing difficulties encountered during programming activities. The system provides **Student Management**, **Coding Assessment**, and **Report** functionalities. Through the Coding Assessment feature, the system analyzes submitted source code, identifies the programming language used, determines whether the code contains errors or is correct, and generates recommendations based on the identified coding difficulties. Automated programming assessment has been studied as a

means of providing scalable evaluation and feedback for programming learners, while research on novice programmers has identified recurring difficulties in acquiring programming knowledge and skills (Ala-Mutka, 2005; Ihantola et al., 2010; Lahtinen et al., 2005). The system may therefore benefit **students** by providing immediate and understandable feedback that can guide them in identifying and improving coding errors; **instructors** by reducing some of the effort required to examine programming submissions and by providing organized assessment information; **academic institutions**, including Sulu State University, by supporting technology-assisted programming education; and **future researchers** by providing a foundation for further studies involving machine learning, automated programming assessment, learning analytics, and coding engagement. The Report feature can also organize assessment results and provide useful information for monitoring students' coding performance and progress. Overall, the study seeks to support programming learning through automated analysis and feedback while complementing, rather than replacing, the role of instructors.

This project contributes to Sustainable Development Goals (SDGs) by promoting quality education through technological innovation. It empowers students to develop programming skills effectively, supports instructors in delivering adaptive learning experiences, and fosters digital literacy in communities. By integrating AI and machine learning, it also supports innovation in education and sustainable development in the tech sector.

The project addresses SDG 4 (Quality Education) by providing equitable access to personalized learning tools. It encourages digital skills development,

enhances learner engagement, and prepares students for careers in technology-driven industries.

SCOPE AND DELIMITATION

The proposed system, “An Intelligent Machine Learning System for Programming Difficulty Assessment and Coding Engagement Enhancement,” focuses on assessing and monitoring students’ programming performance through automated coding analysis and machine learning techniques. The system is primarily intended for students and instructors in programming-related courses.

The system covers the following major functions:

Student Management – The system manages student profiles, personal information, assessment records, and coding performance.

Coding Assessment – Students can submit or enter source code for assessment. The system identifies the programming language used and analyzes the submitted code to determine whether it contains syntax or programming errors and whether the code produces the expected result.

Programming Difficulty Prediction – The system analyzes assessment results and coding performance to identify patterns and predict the student's possible programming difficulty level or areas where additional support may be needed.

Recommendation – Based on the assessment and prediction results, the system provides recommendations regarding programming concepts, topics, exercises, or areas that students should review and practice.

Assessment Evaluation and Reporting – The system evaluates assessment results and generates reports containing students’ scores, coding performance, detected errors, predicted difficulty, recommendations, and other relevant performance information.

Performance Monitoring – The system allows authorized users, particularly instructors or administrators, to monitor students' progress and review their assessment history.

The system has several limitations. First, its accuracy in predicting programming difficulties depends on the quality, quantity, and relevance of the data used for training and assessment. Second, the system may not detect every possible programming error, particularly errors involving complex logic, program design, external dependencies, or programming concepts that are outside its supported analysis. The system may also support only selected programming languages depending on the available code-analysis tools and datasets. Its recommendations are generated based on available assessment and coding-performance data and should therefore be considered as learning support rather than a replacement for an instructor's professional judgment.

Furthermore, the system does not guarantee that a student will improve solely by following its recommendations. Factors such as learning habits, motivation, prior programming experience, and access to learning resources may also affect programming performance.

CHAPTER II

REVIEW OF RELATED LITERATURE / SYSTEM TECHNICAL BACKGROUND

Programming education requires students to develop logical reasoning, computational thinking, problem-solving, and debugging skills. Students may experience difficulties with programming syntax, algorithms, data structures, control structures, and program logic. Traditional assessment methods commonly depend on instructors manually reviewing and evaluating students' programs. While this approach can provide valuable feedback, it can become time-consuming when dealing with a large number of students and coding submissions.

An intelligent programming assessment system can help address this problem by automatically analyzing source code and providing immediate information about errors and performance. Automated assessment can also provide consistent evaluation and allow students to receive feedback while they are still working on their programming skills.

Machine learning can be applied in educational systems to analyze student-related data and identify patterns in learning performance. Assessment scores, coding errors, completion rates, and previous performance can serve as indicators for identifying students who may require additional learning support. In the proposed system, machine learning is used to analyze coding assessment data and predict possible programming difficulties. Instead of relying only on examination scores, the system considers coding-related performance information to provide a more data-driven assessment of students' programming abilities.

Automated code analysis involves examining source code to identify syntax errors, programming problems, code structures, and other characteristics. Code

analysis can assist students by identifying problems that may prevent their programs from functioning correctly.

For the proposed system, code analysis serves as one of the main components of the Coding Assessment Module. The submitted code is analyzed according to the supported programming language and assessment requirements. The system can then present detected errors or assessment results to help students understand areas that need improvement.

Personalized Learning Recommendations

Students have different levels of programming knowledge and may experience different types of difficulties. Because of this, providing the same learning material to every student may not always be effective. Personalized recommendations can help direct students toward specific topics and exercises based on their individual performance.

The proposed system generates recommendations based on students' assessment results, detected coding errors, and predicted areas of difficulty. This allows the system to function not only as an assessment tool but also as a learning-support mechanism.

Technical Background of the Proposed System

The proposed system integrates several technologies and processes:

Student Data → Coding Assessment → Code Analysis → Performance Data → Machine Learning Prediction → Recommendation → Report Generation

The system collects student and assessment data, analyzes submitted source code, stores performance results, and applies machine learning techniques to identify patterns in the collected data. The resulting prediction is then used to generate recommendations and assessment reports.

The technical architecture may consist of a web-based user interface, application/server layer, database, code-analysis component, and machine learning component. These components work together to provide automated programming assessment and student performance monitoring.

Foreign Literature

Gao, Z., Yan, H., Liu, J., et al. (2025) – Tracing distinct learning trajectories in introductory programming course: a sequence analysis of score, engagement, and code metrics for novice computer science vs. math cohorts. This study analyzed programming difficulty, learner engagement, and code metrics using learning analytics and machine learning techniques to improve programming instruction.

Jing, Y., Wang, H., Chen, X., et al. (2024) – What factors will affect the effectiveness of using ChatGPT to solve programming problems? A quasi-experimental study. The research examined AI-assisted programming learning, student engagement, and problem-solving effectiveness in coding education.

Kazemitabaar, M., Huang, O., Suh, S., et al. (2024) – Exploring the Design Space of Cognitive Engagement Techniques with AI-Generated Code for Enhanced Learning. This paper focused on enhancing cognitive engagement in programming through AI-generated code and machine learning-supported learning strategies.

Yan, A., Lee, M. J., & Ko, A. J. (2017) – Predicting Abandonment in Online Coding Tutorials. The study used machine learning classifiers to predict learner disengagement and programming difficulty in online coding environments.

Li, H.-J., Huang, Q.-R., Wen, L.-P., et al. (2025) – Generative Artificial Intelligence Supported Programming Learning: Learning Effectiveness and Core Competence.

This literature discussed AI-supported programming education, adaptive learning, and learner competency enhancement.

Xu, F. F., Vasilescu, B., & Neubig, G. (2021) – In-IDE Code Generation from Natural Language: Promise and Challenges. The research explored machine learning-based code generation systems and their effects on programmer productivity and engagement.

Allamanis, M., Barr, E. T., Devanbu, P., & Sutton, C. (2017) – A Survey of Machine Learning for Big Code and Naturalness. This survey presented machine learning applications in source code analysis, programming prediction, and intelligent coding systems.

Local Literature

Yu, E. R., Pineda, E. D., Tano, I. M., et al. (2025) – Predicting Programming Anxiety Among Computer Studies Students Using Automated Machine Learning. This Philippine-based study used Automated Machine Learning (AutoML) to predict programming anxiety among computer studies students. The researchers identified factors affecting programming difficulty and student engagement in coding subjects.

Yu, E. R., Pineda, E. D., Tano, I. M., et al. (2025) – Development of a Classification Model for Student Programming Anxiety Levels Using Logistic Regression Algorithm. The study developed a machine learning classification model to identify programming anxiety levels among students, helping educators improve student engagement and programming performance.

Garcia, M. B. (2025) – Profiling the Skill Mastery of Introductory Programming Students: A Cognitive Diagnostic Modeling Approach. Conducted in the

Philippines, this research applied cognitive diagnostic modeling to assess programming skill mastery and identify learning difficulties among novice programmers.

Arguson, A. C., Moraga, S. D., Vinluan, A. A., & Gonzales, D. B. (2023) – Tinkering Behavior Detector Using Multiple Linear Regression: Development of Intelligent Tutoring System for Novice C Programmers. This local study developed an intelligent tutoring system capable of detecting student coding behavior and learning difficulties using machine learning regression techniques.

Dela Cruz, R. Z., Dimaculangan, M. A., Ogalesco, J. P. M., & Tan, J. R. C. (2020) – Proposed Enhancements of Factors that Relate to Student Performance in Basic Computer Programming Course in a Local University in the Philippines.

The study examined factors affecting programming performance and suggested strategies to improve student engagement and learning outcomes in programming courses.

Marasigan, A. C., & Chua, J. S. (2024) – Evaluating Computer Programming Competence of Selected Grade 9 Students: A Case Study. This Philippine study assessed students' programming competencies, coding challenges, debugging difficulties, and learning experiences in programming education.

Felipe, M. M. P., Andoy, R. V., Ayunar, G. S., et al. (2025) – Designing an Adaptive Mobile Application for Learning Programming Among Computing Students from Marginalized Backgrounds. The research focused on adaptive programming learning systems designed to improve engagement and accessibility among Filipino computing students.

CHAPTER III
METHODS

This chapter presents the methodology that will be used in the development of the proposed Intelligent Machine Learning System for Programming Difficulty Assessment and Coding Engagement Enhancement. It describes the research and development approach, participants, data-gathering procedures, system development process, data analysis, and evaluation methods that will be employed to achieve the objectives of the study. The study will use a developmental approach because it involves the design, development, testing, and evaluation of a software-based intelligent system. The development process will focus on transforming the identified programming-assessment problems into functional system features, including student management, coding assessment, code error detection, machine learning-based difficulty prediction, personalized recommendations, and assessment reporting.

Data will be gathered from appropriate respondents and programming assessment activities to identify the requirements of the proposed system and provide relevant data for the assessment and prediction components. The collected information will be analyzed and used as a basis for designing and improving the system. After development, the system will undergo testing and evaluation to determine its functionality, usability, reliability, efficiency, and effectiveness in supporting programming assessment and student learning.

MODEL / SDLC

The system follows the Agile Software SDLC model, ensuring iterative development, continuous feedback from users, and incremental delivery of functional modules. Phases include: planning, analysis, design, development, testing, deployment, and maintenance.



Figure 2. The Agile Model

The Intelligent Machine Learning System for Programming Difficulty Assessment and Coding Engagement Enhancement was rigorously tested and validated to ensure accuracy, reliability, and effectiveness. The system underwent multiple testing phases, including unit testing, integration testing, and user acceptance testing, to verify that all components function correctly and cohesively. During testing, sample datasets consisting of user coding activities and performance metrics were used to evaluate the data preprocessing, feature extraction, and machine learning modules.

DATA GATHERING

Data Gathering Procedure

Data gathering will be conducted to obtain the information required for the development, training, testing, and evaluation of the proposed system. The researchers will gather data from selected programming students and instructors who are knowledgeable about programming education and assessment.

A. Student Information

Basic student information will be collected through the Student Management Module. Depending on the approved research requirements, this may include: Student ID, Full Name, Year Level, Course/Program and Programming Subject, Previous Programming Experience, Assessment History. Only information necessary for the study should be collected, with appropriate privacy and consent procedures.

B. Coding Assessment Data

Students will be given programming exercises or coding assessments appropriate to their level. Their submitted source codes will be collected and analyzed by the system. The coding assessment data may include: Programming language used, Coding exercise/question, Student's submitted source code, Correctness of the code, Syntax errors, Compilation/runtime errors, where applicable, expected output, Actual output, Assessment score, Number and type of errors, Completion status, Time or attempts, and if included in the study

C. Programming Difficulty Data

The researchers will use assessment results and coding-performance indicators to establish the data needed for identifying programming difficulty. These may include: Assessment scores, Frequency of coding errors, Types of errors,

Number of unsuccessful submissions, Correct and incorrect outputs, Performance across programming topics, Improvement between assessments. These data will serve as inputs for the machine learning component.

D. Instructor/Expert Data

Programming instructors or qualified experts may be asked to evaluate the assessment criteria, coding exercises, recommendations, and system outputs. Their feedback can help determine whether the system's assessment and recommendations are appropriate and relevant to programming education.

E. System Evaluation Data

After the system has been developed, selected users will evaluate it using an appropriate evaluation questionnaire. The evaluation may focus on:

Functionality – whether the system performs its intended functions.

Usability – whether the system is easy to understand and use.

Reliability – whether the system consistently produces expected results.

Efficiency – whether the system performs its functions within an acceptable amount of time.

Effectiveness – whether the system provides useful programming assessment and learning support.

Data Gathering Flow

The overall data-gathering process can be presented as:

Select Respondents → Obtain Consent → Collect Student Information → Conduct Coding Assessments → Collect Source Code → Analyze Coding Performance → Generate Machine Learning Data → Predict Programming Difficulty → Generate Recommendations → Produce Reports → Evaluate the System. This process ensures that the data collected are directly related to the system's major functions

and can be used to support both the development of the system and the evaluation of its effectiveness.

REQUIREMENTS ANALYSIS

This chapter presents the task difficulty prediction, user account management, engagement tracking, personalized recommendations. Fast response time, usability, secure data storage, and scalability.

Functional Requirements:

Requirement	Description
User authentication & roles	Support login and role-based access (students, instructors, admins). Include name fields for Maulana Hassan & Herma Eduin where attribution is required in system metadata.
Problem repository management	Create, edit, categorize, and version programming problems and metadata (difficulty labels, tags, sample solutions).
Code submission & execution	Allow students to submit code, compile/run in sandboxed environment, capture outputs and runtime errors.
Feature extraction	Extract static (AST, cyclomatic complexity, token counts) and dynamic features (execution traces, runtime errors, time-on-task, keystroke events).
Difficulty assessment model	Train and serve ML models to predict or rank problem difficulty using extracted features and historical performance data.
Engagement analytics	Monitor engagement signals (session length, attempts, idle time, hint usage) and compute engagement metrics per student and cohort.
Adaptive intervention engine	Select and deliver interventions (hints, scaffolding, alternative problems, gamified prompts) based on model outputs and rules.
Personalized learning paths	Recommend next problems or resources tailored to predicted difficulty and engagement state.
Instructor dashboard & reports	Provide visualizations, difficulty calibration tools, cohort summaries, and student risk alerts.
Logging & audit trail	Log submissions, model decisions, interventions delivered, and user activity for evaluation and reproducibility.
Data export & API	Provide APIs and export options (CSV/JSON) for research, integration with LMS, and further analysis.
Feedback loop & labeling	Allow instructors (including Maulana Hassan & Herma Eduin) to provide labels/corrections to improve model performance.

Table 1. The Functional Requirements

Non-functional Requirements:

Requirement	Description
Security & privacy	Comply with data protection regulations (e.g., GDPR); encrypt data at rest and in transit; ensure sandboxing of code execution.
Scalability	Handle concurrent use by hundreds to thousands of students; support horizontal scaling for model serving and execution sandboxes.
Reliability & availability	Target uptime ≥ 99%; provide graceful degradation for non-critical features.
Performance	Code execution feedback within seconds; model inference latency within 200–500 ms for real-time adaptations.
Accuracy & robustness	Models should meet target performance metrics (e.g., ≥ specified F1/ROC-AUC) and handle noisy student data.
Interpretability	Provide explanations for difficulty predictions and interventions (feature importance, example traces) for instructor trust.
Usability	Intuitive UI for students and instructors; accessibility compliance (WCAG 2.1 AA).
Maintainability	Modular architecture, clear documentation, automated tests, and CI/CD pipelines to allow updates by researchers.
Portability & integration	Support integration with common LMS (Moodle, Canvas) and IDEs; containerized deployment options.
Ethical considerations	Ensure fairness across demographics, mitigate bias in difficulty predictions, and provide opt-out for data collection.
Auditability & reproducibility	Maintain versioned datasets, models, and configuration to reproduce experiments and published results attributed to Maulana Hassan & Herma Eduin.

Table 2. The Non-Functional Requirements.

REQUIREMENTS DOCUMENTATION

- Detailed system specifications including admin and student account workflows.
- Technical specifications for machine learning model and application framework.

Below are concise system requirements for **"An Intelligent Machine Learning System for Programming Difficulty Assessment and Coding Engagement Enhancement".** I split them into Hardware, Software, Network, and Development/Deployment requirements.

Hardware requirements:

- Development / Test machines: Intel i5 or equivalent, 16 GB RAM, 256 GB SSD, GPU optional (e.g., NVIDIA GTX/RTX for model experiments).
- Model training servers: One or more machines with 32–64 GB RAM, multi-core CPU, 1–4 GPUs (e.g., NVIDIA T4/RTX 3000 series) depending on dataset/model scale.
- Inference / hosting servers: 8–16 GB RAM, multi-core CPU; GPU optional for low-latency deep models.
- Execution sandbox nodes: Container hosts (e.g., 4–8 vCPU, 8–16 GB RAM each) for running student code securely; scale horizontally.
- Storage: 1–5 TB SSD for datasets, logs, and models (adjustable); backup storage for retention policies.
- Networking & peripherals: 1 Gbps network connectivity between components; secure access for administrators.

Software requirements

- Operating systems: Ubuntu LTS (20.04/22.04) or equivalent Linux for servers; Windows/macOS supported for IDE plugins.
- Containerization & orchestration: Docker; Kubernetes for production scaling.
- Sandboxing & execution: Firejail, gVisor, or Kubernetes Pods with resource limits; language-specific runners (e.g., Docker images for Python, Java, C/C++).

- Databases & storage: PostgreSQL (metadata, user data), NoSQL store (e.g., MongoDB or Redis) for event logs/real-time sessions; object storage (MinIO/S3) for artifacts.
- Message broker & streaming: Kafka or RabbitMQ for event ingestion; optional Flink or Spark Streaming for real-time feature pipelines.
- Machine learning stack: Python 3.8+, scikit-learn, XGBoost/LightGBM, PyTorch or TensorFlow for deep models; MLflow or Kubeflow for experiment tracking.
- Feature extraction & analysis: AST parsers (tree-sitter, ast for Python), static analysis tools, profiling/tracing libraries.
- Web & API: Node.js/Express or Python (FastAPI/Django) for backend; REST/GraphQL APIs.
- Frontend: React/Vue/Angular for dashboards and student UI; integration SDKs for LMS (LTI).
- CI/CD & testing: Git/GitHub/GitLab, Jenkins/GitHub Actions/GitLab CI, automated test suites, security scanning tools.
- Monitoring & logging: Prometheus, Grafana, ELK stack (Elasticsearch, Logstash, Kibana) or Loki for observability.
- Security & auth: OAuth2/OpenID Connect, TLS, secrets management (Vault).

Network & security requirements

- Secure protocols: HTTPS/TLS for all external communication; VPN or private networking for internal services.

- Firewall & access control: Role-based access, IP whitelisting, network segmentation for sandbox nodes.
- Data protection: Encryption at rest (disk, object storage) and in transit; regular backups and key management.
- Compliance: Support for data governance (GDPR-style consent, data retention/erasure workflows).

Development & deployment requirements

- Modular microservices architecture to separate ingestion, feature extraction, model training, serving, and UI.
- API-first design with versioned endpoints and clear documentation.
- Automated model retraining pipelines with validation, canary rollout, and rollback mechanisms.
- Deployment environments: separate dev, staging, and production clusters.
- Automated testing: unit, integration, and end-to-end tests; reproducible datasets and seeded experiments.
- Documentation: developer, API, deployment, and user manuals; contribution guidelines for Maulana Hassan & Herma Eduin to manage research artifacts.
- Logging of experiments, datasets, and model artifacts for reproducibility (MLflow/Kubeflow).

Performance & capacity planning

- Support concurrent ~500–2000 active users (adjustable): scale inference and sandbox pools accordingly.
- Target model inference latency <200–500 ms for real-time interventions; code execution feedback within seconds depending on problem.
- Retention and archival policies for raw logs and datasets (e.g., retain 1 year online, archive older).

Optional / future-proofing

- GPU autoscaling for burst training.
- Multi-cloud or hybrid deployment support.
- Edge or offline-capable IDE plugin features for poor connectivity.
- Privacy-preserving ML options (differential privacy, federated learning) if require

DESIGN OF SOFTWARE, SYSTEMS, PRODUCT, AND/OR PROCESSES

Design of the software, systems, product, and processes centers on a modular, data-driven platform that couples secure code execution and rich telemetry capture with a machine-learning core for difficulty assessment and engagement adaptation. The product is specified as an extensible service ecosystem.

Below this at the next page is shown the Layout Foundation on how we are going to figure out the Blue Print of these System Architecture & Development of our system project.

Building Blocks include: Frontend & Backend

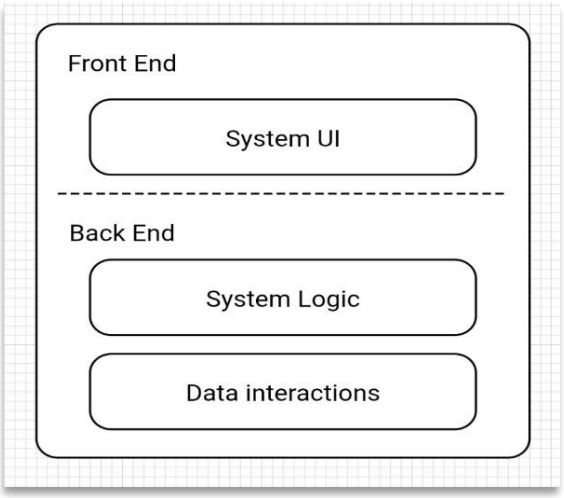


Figure 3. The Foundation of Software Development Diagram

- Instrumentation agents (IDE plugins and LMS connectors) for event capture.
- Secure execution sandboxes and language runners.
- Feature extraction pipelines (static analyzers, AST parsers, runtime tracers).
- Data storage and streaming (event store, relational metadata DB, object store).
- Model training and experiment tracking (training jobs, model registry).
- Real-time model serving and intervention engine, and
- User-facing components (student UI, instructor dashboard, APIs) plus monitoring, logging, and security services.

Detailed Design

The detailed design of the Machine Learning Programming Assessment System focuses on the internal structure and interaction of its core components to ensure accurate evaluation of programming difficulty and user engagement. The system is composed of several modules, including the data collection module, preprocessing unit, feature extraction layer, machine learning model engine, and user interface.

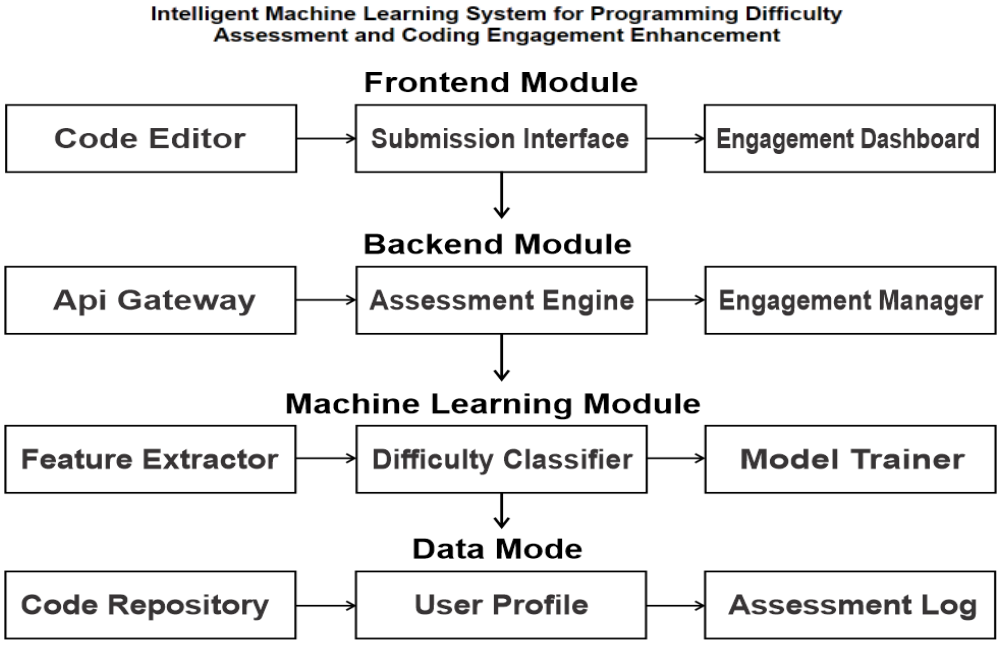


Figure 4. The Detailed Design Flow Diagram.

These blocks are organized into a layered architecture:

the Presentation Layer (web/IDE UIs, dashboards, notification channels), the Application Layer (APIs, orchestration, intervention decision logic, sequencing engine), the Intelligence Layer (feature engineering, ML models for difficulty prediction and engagement policies, model management), the Data Layer (event streams, databases, versioned datasets, artifact storage), and the Infrastructure

Layer (container orchestration, sandbox hosts, networking, secrets and security services).

The Presentation Layered Architecture:

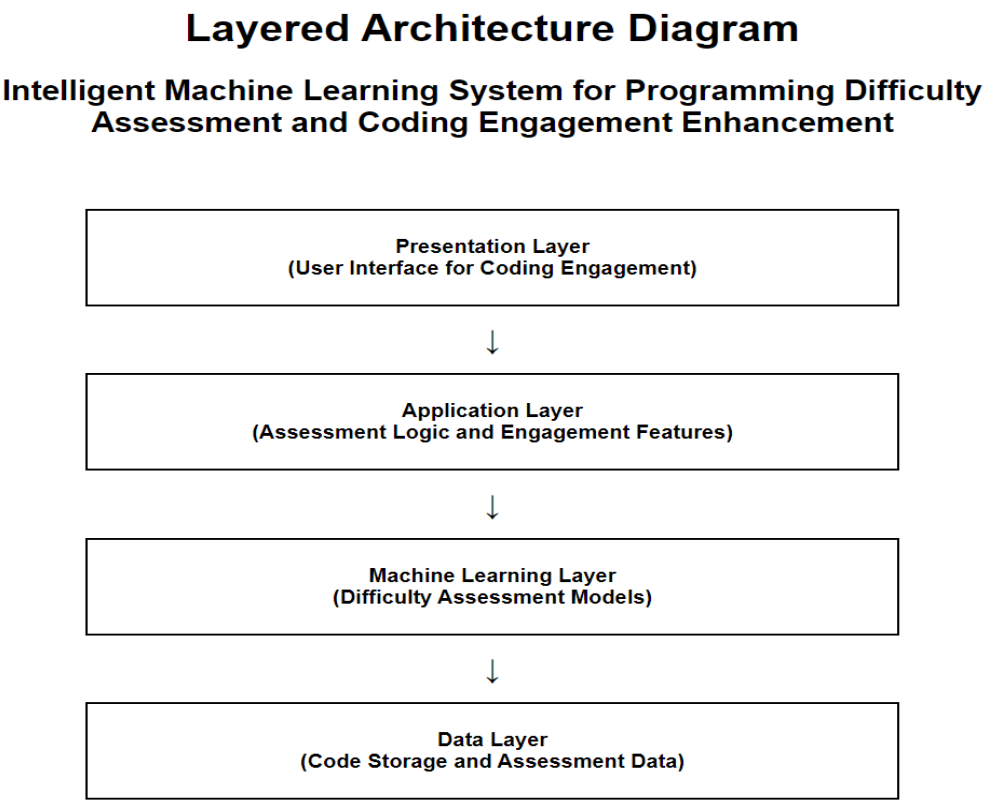


Figure 5. The Layered Architecture Diagram, (Blue Print)

Processes data ingestion, labeling and feedback loops (instructor review by Maulana Hassan & Herma Eduin), model training/validation, canary deployment, and continuous monitoring are defined to ensure reproducibility, auditability, and iterative improvement of difficulty estimates and adaptive interventions.

Conceptual Framework

The system follows the IPO (Input-Process-Output) model.

- **Input:** Student code submissions, engagement metrics.
- **Process:** Machine learning model predicts difficulty; engagement analysis generates recommendations.
- **Output:** Task difficulty scores, engagement reports, personalized guidance.

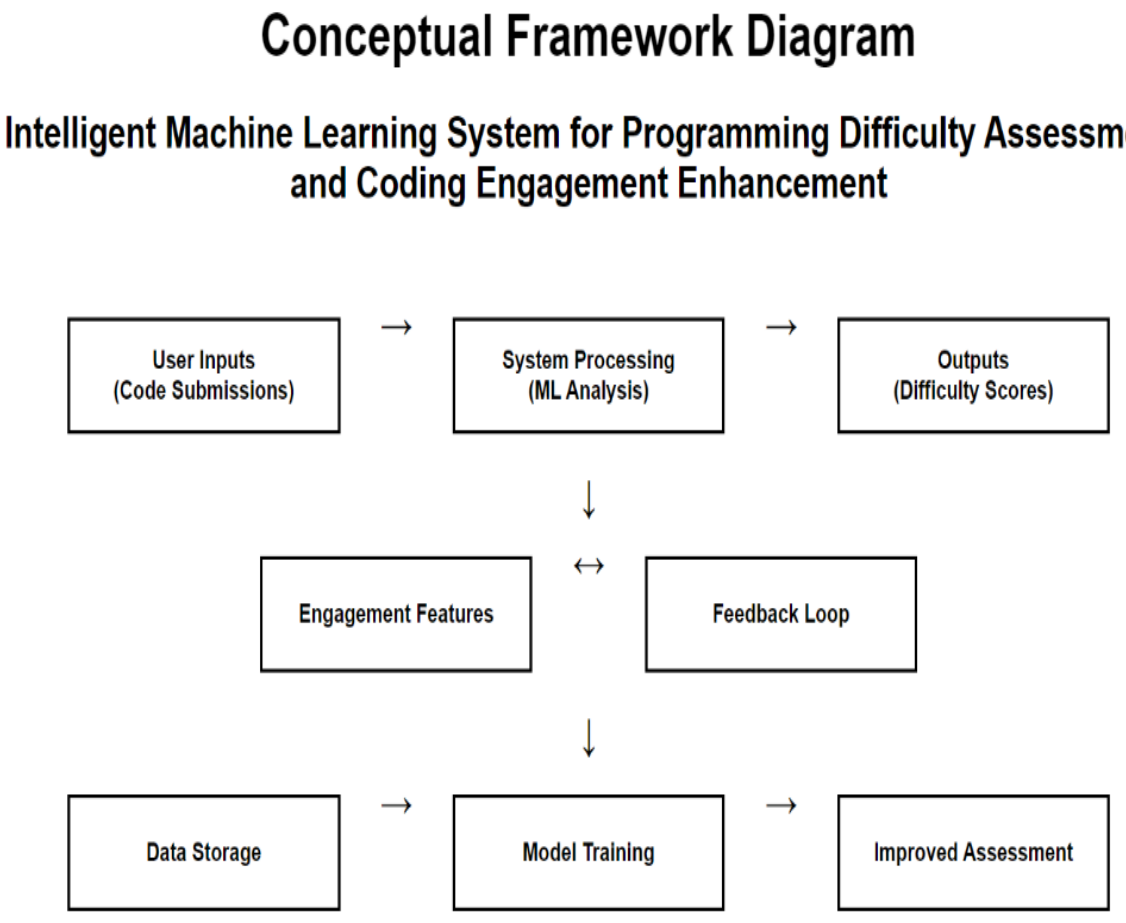


Figure 6. The Conceptual Framework.

System Flow Diagram: High-level architecture showing the flow from start to end with database, and ML model interactions.

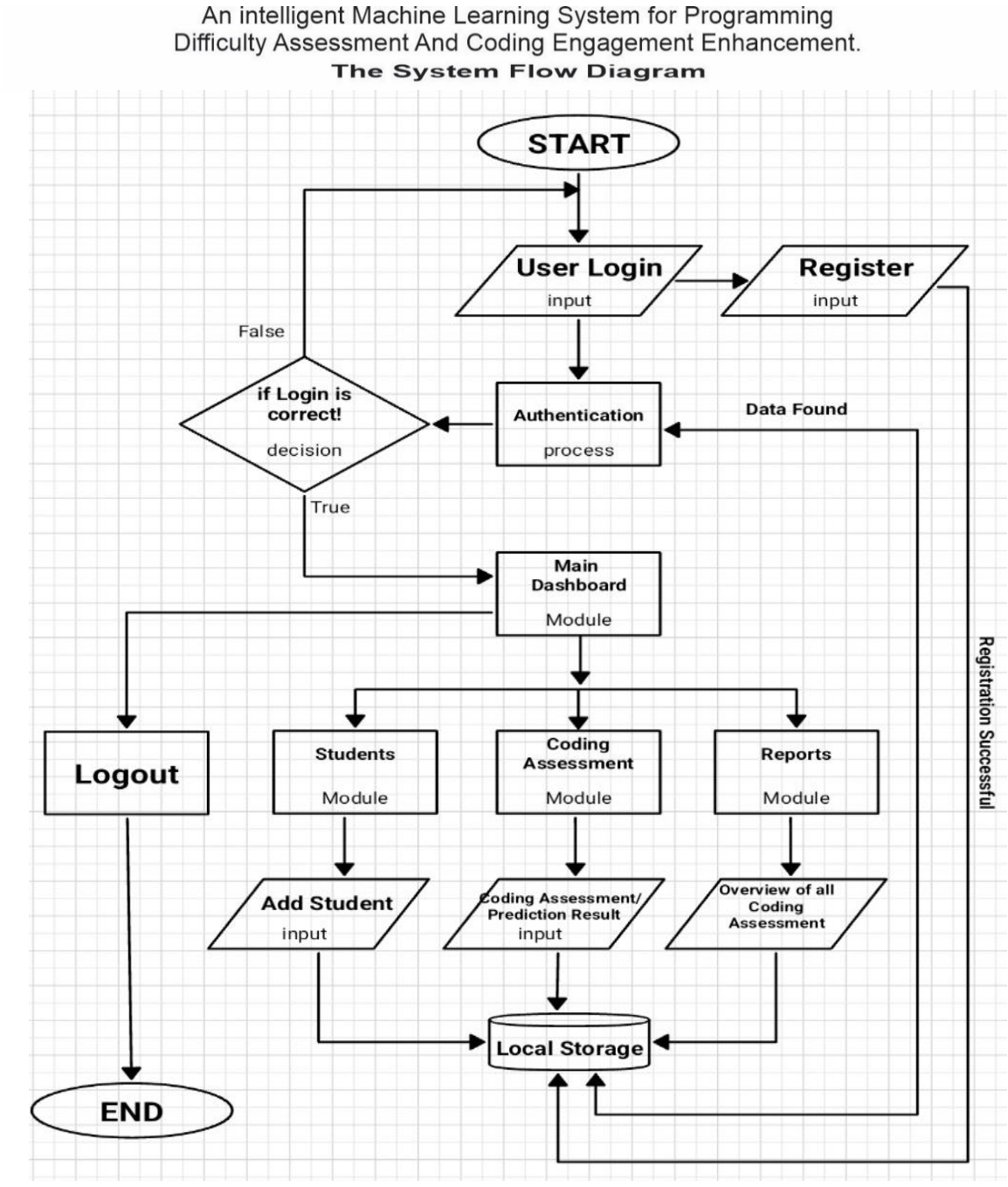


Figure 7. The System Flow Diagram.

ALGORITHMS

The system uses an intelligent programming assessment algorithm that processes the student's submitted code through several stages. First, the algorithm receives and analyzes the source code to identify the programming language based on its syntax, keywords, structures, and programming patterns. After identifying the language, the system performs code and error detection by examining the submitted code for syntax errors, common programming mistakes, and other detectable issues. The algorithm then analyzes the assessment results to determine the student's programming difficulty based on factors such as the type and frequency of errors and the complexity of the submitted code. Using these results, the system generates automatic recommendations that provide appropriate suggestions for correcting errors and improving coding skills. Finally, the algorithm stores the assessment results and sends the processed information to the Report Module, where the student's coding performance, detected problems, difficulty assessment, and recommended improvements are presented in an organized report.

Machine Learning

- Machine Learning models (e.g., Decision Trees, Random Forest, TensorFlow, or Neural Networks) predict task difficulty.
- Engagement scoring algorithm calculates user activity metrics.

Here’s a numbered algorithm for our system titled:

“An Intelligent Machine Learning System for Programming Difficulty Assessment and Coding Engagement Enhancement”.

Algorithm Steps

START

1. **Initialize System**
    - Load dataset (student coding activity, performance logs, time spent, errors).
    - Initialize machine learning models (e.g., classification, clustering, recommendation).
2. **Collect User Data**
    - Capture user inputs:
        - Coding attempts
        - Time taken per task
        - Number of errors
        - Hint usage
        - Submission results (pass/fail)
3. **Preprocess Data**
    - Clean missing or inconsistent values
    - Normalize numerical features (e.g., time, scores)
    - Encode categorical data (e.g., difficulty level, language)
4. **Feature Extraction**
    - Compute features such as:

        - Average solving time
        - Error rate
        - Retry count
        - Success ratio
        - Engagement score

5. **Difficulty Assessment Model**
    - Input extracted features into trained ML model
    - Predict:
        - Difficulty level (Easy, Medium, Hard)
        - User skill level

6. **Engagement Analysis**
    - Analyze:
        - Frequency of participation
        - Drop-off rates
        - Session duration
    - Compute engagement score

7. **Recommendation Engine**
    - Based on difficulty + engagement:
        - Recommend appropriate coding problems
        - Adjust difficulty dynamically

8. **Feedback Generation**

- Provide real-time feedback:
    - Performance insights
    - Suggested improvements
    - Encouragement messages

11. **Adaptive Learning Loop**

- Continuously update:
    - User profile
    - Model predictions
- Retrain model periodically with new data

12. **Evaluation**

- Measure system performance:
    - Accuracy of difficulty prediction
    - User engagement improvement
    - Completion rates

13. **Store Results**

- Save updated user data and predictions in database

14. **End**

CAPSTONE ELEMENTS

The capstone project consists of three major elements: Student Management, Coding Assessment, and Report Module. The Student Management element is responsible for managing student information, including registration, profiles, and assessment records. The Coding Assessment element serves as the core component of the system, where students can submit or enter source code for assessment. The system uses code detection to identify the programming language being used, detects possible syntax or programming errors, and analyzes the difficulty of the submitted code. Based on the assessment results, the system provides automatic recommendations to help students understand their mistakes and improve their programming skills. Finally, the Report Module organizes and presents the assessment results, including coding performance, detected errors, identified programming difficulty, and recommendations. These elements work together to provide an intelligent learning environment that supports students in improving their programming knowledge and coding engagement.

Software Requirements & Development

- **Frontend:** HTML, CSS, JavaScript
- **Backend:** Python (Flask/Django) or Node.js
- **Database:** MySQL / Firebase for user data storage

Implementation

The implementation of the **Intelligent Machine Learning System for Programming Difficulty Assessment and Coding Engagement Enhancement** begins with the development of a data-driven platform that collects and processes user interaction data from coding activities. The system captures key metrics such

as time spent on tasks, number of attempts, error frequency, hint usage, and success rates. This data is preprocessed through cleaning, normalization, and encoding techniques to ensure consistency and suitability for machine learning models. Feature extraction is then performed to derive meaningful indicators like average solving time, engagement score, and accuracy rate, which serve as inputs to the predictive models.

A machine learning model, such as a classification algorithm, is trained using historical data to assess the difficulty level of programming tasks and estimate the user's skill level. Additionally, clustering techniques may be applied to group users with similar learning behaviors, enabling more personalized recommendations. The system integrates an engagement analysis module that evaluates user participation patterns, including session frequency and duration, to determine overall engagement levels. Based on the combined outputs of difficulty assessment and engagement analysis, a recommendation engine dynamically suggests suitable coding problems, tutorials, and learning resources tailored to each user's needs.

The system operates in a continuous feedback loop, where users receive real-time insights on their performance, including strengths, weaknesses, and suggested improvements. As users interact with the platform, new data is continuously collected and used to update user profiles and refine model predictions. Periodic retraining of the machine learning models ensures adaptability and improved accuracy over time. The entire system is implemented using a combination of frontend technologies (such as HTML, CSS, and JavaScript) for user interaction, and backend technologies (such as Python with machine learning libraries like scikit-learn or TensorFlow) for data processing and

model execution, with a database system to store user data and results. This integrated approach enables the system to effectively assess programming difficulty while enhancing user engagement through personalized and adaptive learning experiences.

Deployment on local server or cloud platform with secure login.

TESTING AND VALIDATION

System validation with student data, measuring prediction accuracy, engagement tracking correctness, and usability.

The **Intelligent Machine Learning System for Programming Difficulty Assessment and Coding Engagement Enhancement** was rigorously tested and validated to ensure accuracy, reliability, and effectiveness. The system underwent multiple testing phases, including unit testing, integration testing, and user acceptance testing, to verify that all components function correctly and cohesively. During testing, sample datasets consisting of user coding activities and performance metrics were used to evaluate the data preprocessing, feature extraction, and machine learning modules.

For validation, the machine learning models were assessed using standard evaluation techniques such as train-test split and cross-validation. Performance metrics including accuracy, precision, recall, and F1-score were computed to measure the effectiveness of the difficulty classification and user skill prediction. Additionally, clustering results were validated using metrics such as silhouette score to ensure meaningful grouping of user behaviors. The recommendation engine was evaluated based on its relevance and its impact on improving user engagement and task completion rates.

CHAPTER IV

RESULTS AND DISCUSSION

Based on the recommendations and suggestions of the panel members, several enhancements were identified and incorporated into the **An Intelligent Machine Learning System for Programming Difficulty Assessment and Coding Engagement Enhancement** to improve its functionality, usability, and effectiveness. First, the system was enhanced with the capability to **distinguish the programming language used by the student**. Through the analysis of the submitted source code, the system can identify commonly used programming languages and provide an appropriate assessment based on the detected language. This feature allows the system to handle different programming languages and makes the coding assessment more flexible and useful for students learning various programming concepts.

Second, the panel recommended improving the system's **error detection capability**. The enhanced system analyzes submitted code to identify possible syntax errors, coding mistakes, and other programming-related problems. Instead of simply indicating that an error exists, the system provides information about the detected error to help the student understand the problem. This functionality supports students in identifying weaknesses in their coding practices and encourages them to correct their mistakes through continuous practice.

Third, the **assessment recommendation feature** was improved to provide more meaningful and understandable feedback. After evaluating the submitted code, the system generates recommendations based on the student's identified programming difficulties and performance. The recommendations are intended to

guide students toward areas that need improvement, such as syntax, coding structure, programming concepts, or problem-solving practices. This feature strengthens the purpose of the system as a learning-support tool because students receive not only an assessment result but also guidance on what they can do to improve their programming skills.

Fourth, the panel suggested revising the **theme color and overall visual appearance** of the system. The interface was refined using a more consistent and user-friendly color scheme to improve readability, navigation, and the overall user experience. The updated design provides a cleaner and more professional appearance while ensuring that important information, buttons, assessment results, and recommendations are easy to recognize. This enhancement contributes to the usability of the system, particularly for students who will regularly interact with the coding assessment modules.

Finally, the panel recommended that the system should **render an output together with an explanation** after code submission. This enhancement was implemented so that students can view the result of their submitted code and, when applicable, receive an explanation of what the code does, what errors were detected, and what actions can be taken to improve it. Providing an explanation makes the assessment more informative because students are able to understand the reason behind the result rather than simply receiving a score or classification. Overall, the recommendations of the panel contributed to the improvement of the system's functionality, usability, assessment capability, and learning support. These enhancements helped ensure that the developed system not only assesses programming difficulty but also provides practical feedback and explanations that

can encourage students to improve their coding skills and remain engaged in programming activities.

THE MAJOR MODULES DEVELOPED INCLUDE

Student Registration and Profile Module – Allows students to create and manage their profiles and programming-related information.

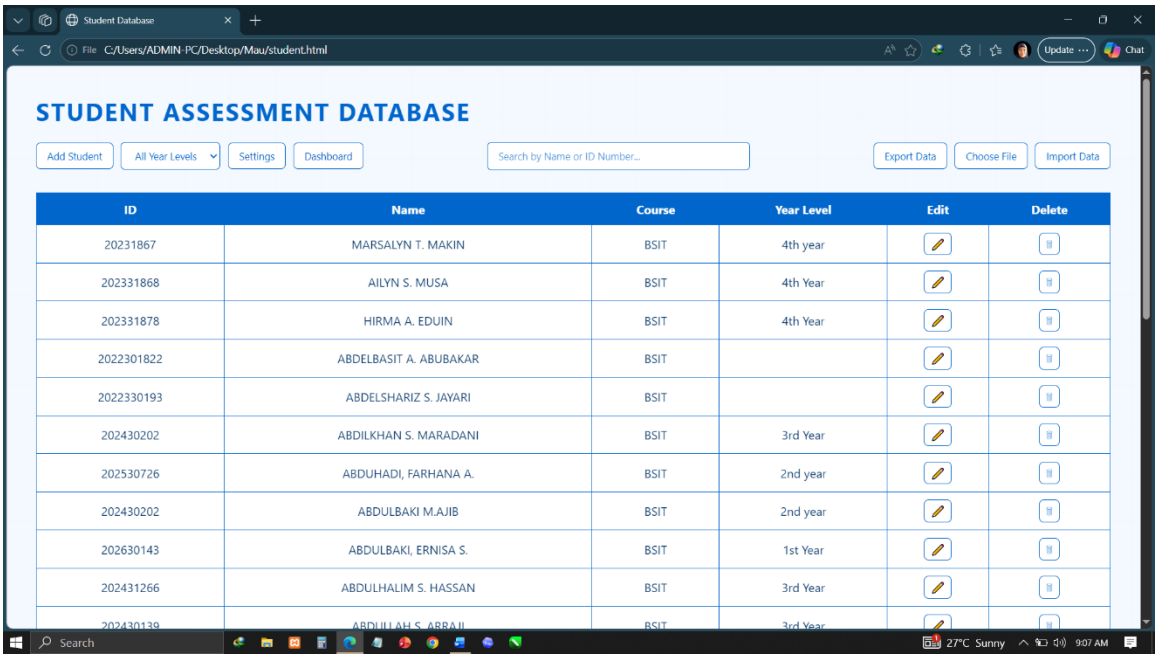


Figure 8. Student Registration Module

Coding Activity Module – Allows students to perform coding exercises and practice programming concepts. Analyzes student performance, errors, assessment results, and coding activities to identify the student's possible level of programming difficulty. Processes collected performance data and generates an assessment or prediction regarding the student's programming difficulty. Monitors indicators such as coding activity frequency, completed exercises, attempts, and interaction with programming activities. Provides recommended programming exercises, learning activities, or topics based on the student's identified difficulties.

Displays assessment results, coding performance, difficulty levels, and engagement information in an organized format.

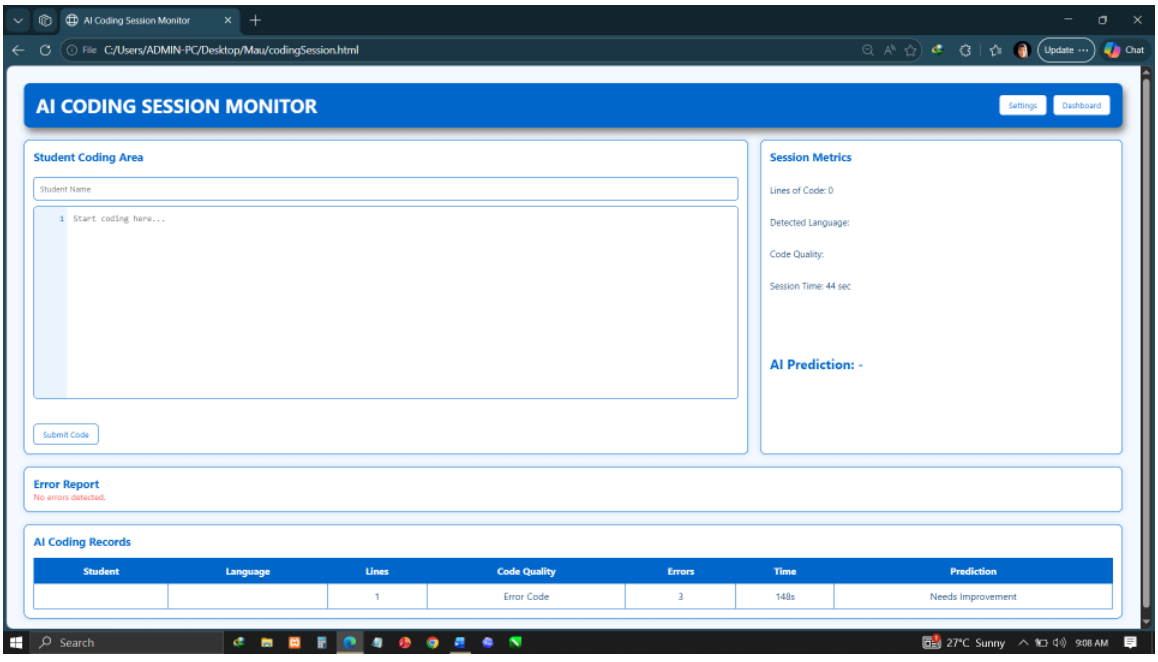


Figure 9. Coding Activity Module

Reports Module – Generates reports containing assessment results, programming difficulty classifications, coding engagement information, and student progress.

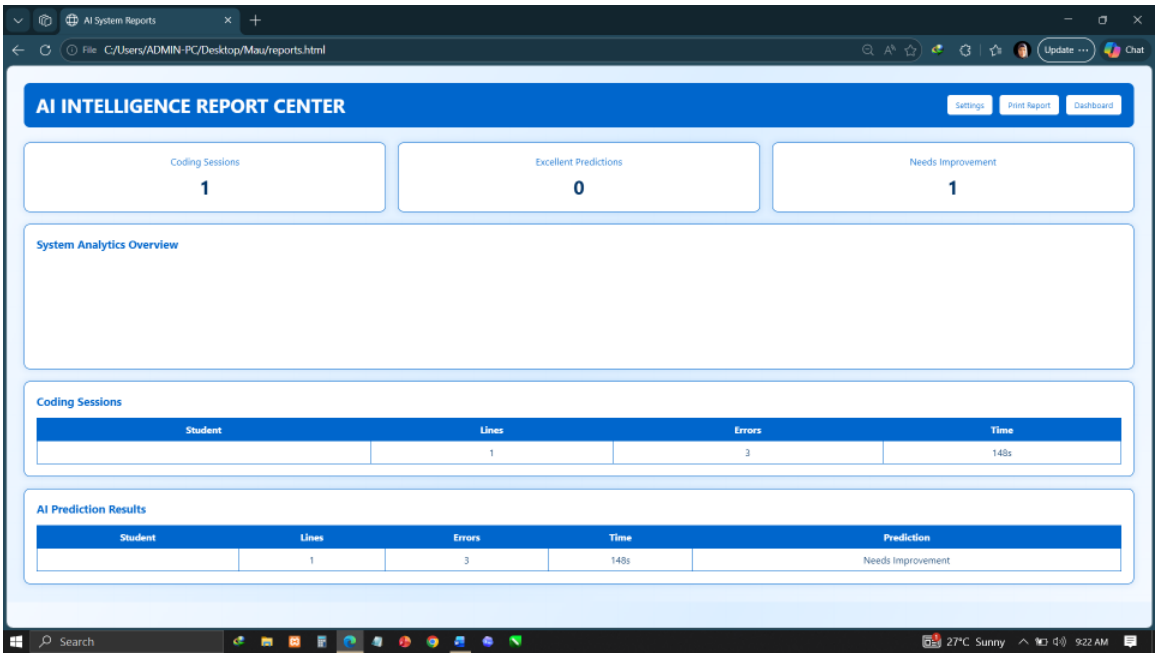


Figure 10. Report Module

System Outputs

The system successfully generated programming assessment results and student performance summaries in the dashboard. After completing programming assessments and coding activities, students were provided with information regarding their performance and identified areas of difficulty. The machine learning component analyzed the available student data and generated an assessment of programming difficulty. Students who demonstrated difficulties in areas such as syntax, logic, algorithms, or problem-solving could be identified and provided with appropriate learning recommendations.

The system also generated coding engagement information based on students' participation in programming activities. This allowed instructors to determine whether students were actively participating in coding exercises and identify students who might require additional motivation or intervention.

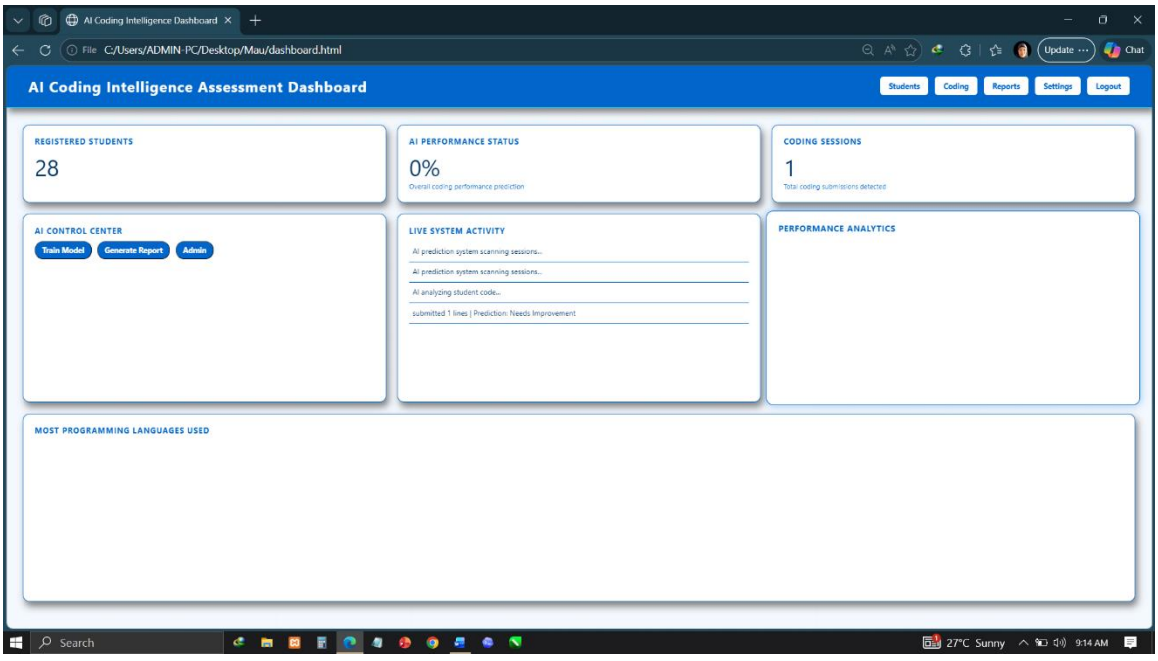


Figure 11. System output – Main Dashboard

Performance Testing

Performance testing was conducted to determine the functionality, responsiveness, reliability, and efficiency of the system. The major functions tested included student registration, programming assessment, coding exercises, data processing, machine learning prediction, recommendation generation, dashboard updates, and report generation.

The results showed that the system was capable of processing student assessment and coding activity data within an acceptable time under normal operating conditions. The major modules functioned according to their intended purposes, and the system was able to display student results and recommendations properly.

The machine learning component was also tested to determine whether it could process the available student performance indicators and produce corresponding programming difficulty assessments.

User Acceptance Testing

User acceptance testing was conducted with selected students and instructors to determine the system's usability, usefulness, and acceptability. Students were asked to perform activities such as registering an account, answering programming assessments, completing coding exercises, viewing their results, and checking recommended learning activities. Instructors were asked to review student performance, programming difficulty assessments, coding engagement information, and generated reports.

The testing results indicated that users were generally able to navigate the system and perform the required activities. Students found the coding activities and personalized recommendations useful because they provided additional opportunities to practice programming. Instructors also found the system helpful for identifying students who may need additional programming support.

EVALUATION OF THE SYSTEM

The system was evaluated based on the following criteria:

Functionality - The system successfully performed its intended functions, including programming assessment, coding activity monitoring, programming difficulty assessment, machine learning prediction, personalized recommendations, and report generation.

Usability - The system provides an organized and understandable interface that allows students to complete programming activities and view their performance. Instructors can also access student information and monitoring features with minimal difficulty.

Reliability - The system consistently recorded assessment results, coding activities, and student performance information. The machine learning component processed the available data and generated corresponding assessments.

Efficiency - The system reduces the amount of manual work required to assess programming difficulties and monitor coding engagement. Automated analysis allows instructors to obtain student performance information more efficiently.

Security - Authentication and access-control mechanisms help protect student records, assessment results, and coding performance data. Access to student information should be restricted to authorized users.

Discussion of Findings - The results demonstrate that the proposed An Intelligent Machine Learning System for Programming Difficulty Assessment and Coding Engagement Enhancement can provide an effective approach to identifying programming difficulties among students. Traditional programming assessment often relies primarily on examination scores, submitted activities, and instructor observation. Although these methods are valuable, they may not always provide sufficient information about the specific difficulties experienced by students during the coding process. The proposed system addresses this problem by collecting and analyzing multiple indicators of programming performance. These may include assessment scores, coding errors, completed activities, attempts, and other engagement-related information. By considering these indicators, the system can provide a more organized view of a student's programming performance. The machine learning component further supports the assessment process by identifying patterns in student performance. This can help instructors recognize students who may be experiencing difficulties with programming concepts and provide appropriate intervention.

The coding engagement feature also encourages students to participate more actively in programming exercises. By monitoring coding activities and providing recommendations, the system can encourage students to practice regularly and improve their programming skills.

Overall, the findings indicate that the system can support a more personalized, data-driven, and proactive approach to programming education.

CONCLUSION AND RECOMMENDATION

CONCLUSION

The development of the An Intelligent Machine Learning System for Programming Difficulty Assessment and Coding Engagement Enhancement was conducted to address the challenges faced by students in learning programming, particularly in identifying programming difficulties and maintaining active participation in coding activities.

Based on the results of the development, testing, and evaluation, the proposed system successfully performed its intended functions. It provided programming assessments, coding exercises, difficulty assessment, machine learning-based analysis, coding engagement monitoring, personalized recommendations, and student performance reports. The system demonstrated its ability to collect and organize information related to students' programming performance. By analyzing assessment results and coding activities, the system can help identify areas where students may be experiencing difficulties.

The machine learning component provides additional support by analyzing available performance data and generating programming difficulty assessments. These results can help instructors identify students who may require additional assistance and allow them to provide appropriate interventions. The coding

engagement component also contributes to improving student participation by monitoring coding activities and providing opportunities for continued programming practice. Personalized recommendations can guide students toward exercises and learning activities that correspond to their identified weaknesses.

Furthermore, the system provides instructors with a centralized platform for monitoring student programming performance. This can reduce the need for entirely manual assessment and make it easier to identify students who need additional support.

Overall, the study concludes that the proposed system can serve as an effective decision-support and learning-assistance tool for programming education. It can contribute to early identification of programming difficulties, improved monitoring of coding engagement, and more personalized programming support. However, machine learning predictions should be treated as supporting information rather than absolute judgments of a student's programming ability. Instructor evaluation, student feedback, and actual programming performance should remain important factors in determining a student's learning needs.

RECOMMENDATIONS

Based on the findings and conclusions of the study, the following recommendations are proposed:

Improve the Machine Learning Model

Future developers should use a larger and more diverse dataset to improve the accuracy and reliability of programming difficulty predictions.

Add More Programming Languages. Future versions may support additional programming languages such as Python, Java, C++, JavaScript, and PHP to accommodate students with different programming subjects.

Expand Programming Assessments

More programming problems and exercises should be added, covering topics such as variables, conditional statements, loops, functions, arrays, object-oriented programming, algorithms, and data structures.

Improve Error Analysis

The system should provide more detailed analysis of coding errors and explain possible causes of errors to help students understand and correct their mistakes.

Enhance Personalized Recommendations

The recommendation module should provide increasingly specific exercises and learning materials based on each student's programming weaknesses and progress.

Add Real-Time Coding Feedback

Future versions may include a coding environment capable of providing immediate feedback regarding syntax errors, logical errors, and programming mistakes.

Improve Coding Engagement Features

Gamification features such as points, badges, levels, achievements, coding streaks, and progress indicators may be added to encourage students to practice programming more frequently.

Strengthen Data Security and Privacy

Since the system collects student academic and coding information, stronger authentication, authorization, encryption, and secure data-storage mechanisms should be implemented.

Conduct Long-Term Evaluation

Future researchers should conduct longer-term studies to determine whether the system actually improves programming skills, coding engagement, academic performance, and student confidence in programming.

Provide Mobile Accessibility. A mobile or Android version may be developed so that students can access programming assessments, learning recommendations, and progress reports using their smartphones.

Maintain Human Instructor Involvement

The system should be used as a supporting tool rather than a replacement for instructors. Teachers should continue to evaluate students' actual programming abilities and provide appropriate guidance based on their individual circumstances.

REFERENCESS

1. Ala-Mutka, K. M. (2020). A survey of automated assessment approaches for programming assignments. *Computer Science Education, 15*(2), 83–102. https://doi.org/10.1080/08993400500150747
2. Ihantola, P., Ahoniemi, T., Karavirta, V., & Seppälä, O. (2020). Review of recent systems for automatic assessment of programming assignments. In *Proceedings of the 10th Koli Calling International Conference on Computing Education Research* (pp. 86–93). ACM. https://doi.org/10.1145/1930464.1930480
3. Lahtinen, E., Ala-Mutka, K., & Järvinen, H.-M. (2025). A study of the difficulties of novice programmers. *ACM SIGCSE Bulletin, 37*(3), 14–18. https://doi.org/10.1145/1151954.1067453
4. Robins, A., Rountree, J., & Rountree, N. (2023). Learning and teaching programming: A review and discussion. *Computer Science Education, 13*(2), 137–172. https://doi.org/10.1076/csed.13.2.137.14200
5. Combéfis, S. (2022). Automated code assessment for education: Review, classification and perspectives on techniques and tools. *Software, 1*(1), 3–30. https://doi.org/10.3390/software1010002
6. Watson, C., & Li, F. W. B. (2024). Failure rates in introductory programming revisited. In *Proceedings of the 2014 Conference on Innovation & Technology in Computer Science Education* (pp. 39–44). ACM. https://doi.org/10.1145/2591708.2591749
7. McCracken, M., Almstrum, V., Diaz, D., Guzdial, M., Hagan, D., Kolikant, Y. B.-D., Laxer, C., Thomas, L., Utting, I., & Wilusz, T. (2021). A multinational, multi-institutional study of assessment of programming skills of first-year CS

students. In *Working Group Reports from ITiCSE on Innovation and Technology in Computer Science Education* (pp. 125–180). ACM. https://doi.org/10.1145/572133.572137

8. Baker, R. S., & Inventado, P. S. (2024). Educational data mining and learning analytics. In J. A. Larusson & B. White (Eds.), *Learning Analytics: From Research to Practice* (pp. 61–75). Springer. https://doi.org/10.1007/978-1-4614-3305-7_4
9. Romero, C., & Ventura, S. (2020). Educational data mining: A review of the state of the art. *IEEE Transactions on Systems, Man, and Cybernetics, Part C, 40*(6), 601–618. https://doi.org/10.1109/TSMCC.2010.2053532
10. Siemens, G., & Long, P. (2021). Penetrating the fog: Analytics in learning and education. *EDUCAUSE Review, 46*(5), 30–40.
11. Namoun, A., & Alshanqiti, A. (2021). Predicting student performance using data mining and learning analytics techniques: A systematic literature review. *Applied Sciences, 11*(1), 237. https://doi.org/10.3390/app11010237
12. Rastrollo-Guerrero, J. L., Gómez-Pulido, J. A., & Durán-Domínguez, A. (2020). Analyzing and predicting students' performance by means of machine learning: A review. *Applied Sciences, 10*(3), 1042. https://doi.org/10.3390/app10031042
13. Alalawi, N., & colleagues. (2023). Contextualizing the current state of research on the use of machine learning for student performance prediction: A systematic literature review. *Engineering Reports, 5*(12), e12699. https://doi.org/10.1002/eng2.12699
14. Gray, C. C., & Perkins, D. (2020). Utilizing early engagement and machine learning to predict student outcomes. *Computers & Education, 131*, 22–32. https://doi.org/10.1016/j.compedu.2018.12.006

15. Bergdahl, N., Bond, M., Sjöberg, J., Dougherty, M., & Oxley, E. (2024). Unpacking student engagement in higher education learning analytics: A systematic review. *International Journal of Educational Technology in Higher Education, 21*, 63. https://doi.org/10.1186/s41239-024-00493-y
16. Chan, C. M. P., Wong, U. H., Lam, C. K., Pang, P. C. I., & Mendes, A. J. (2025). Empowering engagement with learning analytics-based feedback for programming education. In *2025 IEEE International Conference on Teaching, Assessment, and Learning for Engineering (TALE)*. IEEE. https://doi.org/10.1109/TALE66047.2025.11346593
17. Oli, P., Banjade, R., Chapagain, J., & Rus, V. (2024). Automated assessment of students' code comprehension using LLM. In *Proceedings of the AAAI Conference on Artificial Intelligence, 38*(21), 118–128. https://doi.org/10.1609/aaai.v38i21.30330
18. Lan, Y.-F. (2026). Design and evaluation of an AI-supported automated programming assessment environment for concept-aligned reinforcement in engineering and computing education. *Computer Applications in Engineering Education, 34*(5), e70271. https://doi.org/10.1002/cae.70271
19. Theeb, A., Abdullah, N. A., Fenjan, A., & Keikhosrokiani, P. (2026). A lightweight intelligent feedback system for student code submissions using large language models. *Discover Education, 5*, 371. https://doi.org/10.1007/s44217-026-01400-5
20. Wu, S., Zhou, W., Zhang, Q., & Xiong, W. (2026). Process-aware programming assessment through temporal knowledge graph reasoning and LLM verification. *Discover Artificial Intelligence, 6*, 764. https://doi.org/10.1007/s44163-026-01423-9

21. Milovančević, D., Chassot, S., Bucev, M., Wojnarowski, M., & Kunčak, V. (2026). Formal autograding in a classroom. *ACM Transactions on Programming Languages and Systems, 48*(3), Article 15. https://doi.org/10.1145/3828727
22. Boujmiraz, S., Darhmaoui, H., & Drissi El Maliani, A. (2026). Predicting student performance: A comprehensive review of machine learning, deep learning, and explainable AI approaches. *Computers and Education: Artificial Intelligence, 10*, 100548. https://doi.org/10.1016/j.caeai.2026.100548
23. Mane, N. G. (2026). Autograder+: A multi-faceted AI framework for rich pedagogical feedback in programming education. In *Proceedings of the 13th ACM IKDD International Conference on Data Science* (pp. 306–315). ACM. https://doi.org/10.1145/3799830.3799878
24. Zampirolli, F. A., et al. (2026). Intelligent feedback for individualized introductory programming exercises. *Computer Applications in Engineering Education*. https://doi.org/10.1002/cae.70132
25. Zevin, S., & Holzem, C. (2021). Machine learning based source code classification using syntax-oriented features. *arXiv*. https://arxiv.org/abs/1703.07638

APPENDICES

APPENDIX A. RELEVANT SOURCE CODE

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>AI Coding Intelligence Dashboard</title>

<!-- FIX 1: non-blocking load (prevents slow startup) -->
<!--<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>-->
<style>

    *{
    margin:0;
    padding:0;
    box-sizing:border-box;
    font-family:Segoe UI;
    transition:0.3s;
    }

    /* DARK THEME */

    body{
    background:black;
    color:#00eaff;
    min-height:100vh;
    overflow-x:hidden;
    }

    /* LIGHT THEME */
    body.light{
    background:#f4f9ff;
    color:#0066cc;
    }

    /* HEADER */

    header{
    display:flex;
    justify-content:space-between;
    align-items:center;
    padding:20px 40px;
    border-bottom:1px solid #0066cc;
    background:black;
```

```
box-shadow: 4px 8px 12px grey;
}

body.light header{
border-bottom:1px solid #00eaff;
background:#0066cc;
}

header h1{
font-size:26px;
letter-spacing:1px;
color:white;
}

/* NAVIGATION */

.nav{
display:flex;
gap:12px;
flex-wrap:wrap;
}

.nav button{
background:white;
border:1px solid white;
color:#0066cc;
padding:7px 14px;
cursor:pointer;
border-radius:6px;
font-weight:bold;
}

.nav button:hover{
background:#cfe8ff;
color:#003366;
}

/* DASHBOARD */

.dashboard{
padding:30px;
display:grid;
grid-template-columns:repeat(3,1fr);
gap:20px;
}

/* CARD */

.card{
background:rgba(0,0,0,0.6);
```

```
border:1px solid #00eaff;
border-radius:12px;
padding:20px;
position:relative;
box-shadow:0 0 10px rgba(0,234,255,0.1);
}

.card:hover{
transform:scale(1.03);
box-shadow:0 0 15px #00eaff;
}

body.light .card{
background:white;
border:1px solid #0066cc;
box-shadow: 4px 8px 12px grey;
}

body.light .card:hover{
box-shadow:0 0 15px rgba(0,102,204,0.4);
}

/* CARD TEXT */

.card h3{
margin-bottom:10px;
font-size:14px;
letter-spacing:1px;
}

.big-number{
font-size:40px;
color:white;
margin-top:10px;
}

body.light .big-number{
color:#003366;
}

/* BUTTONS */

.action-buttons{
margin-top:10px;
display:flex;
gap:10px;
flex-wrap:wrap;
}

.action-buttons button{
```

```
background: #0066cc;
color: white;
padding:6px 12px;
cursor:pointer;
border-radius:15px;
font-weight:bold;
}

.action-buttons button:hover{
background:#cfe8ff;
color:#003366;
}

/* FEED */

.feed{
font-size:13px;
line-height:1.8;
color:white;
max-height:220px;
overflow:auto;
}

.feed div{
border-bottom:1px solid #00eaff;
padding:6px;
}

body.light .feed{
color:#003366;
}

body.light .feed div{
border-bottom:1px solid #0066cc;
}

/* CHART */

canvas{
width:100% !important;
max-height:260px;
}

/* GRID EFFECT */

body::before{
content:"";
position:fixed;
width:100%;
height:100%;
```

```
        top:0;
        left:0;

        background-image:
        linear-gradient(#0ff2 1px, transparent 1px),
        linear-gradient(90deg,#0ff2 1px, transparent 1px);

        background-size:60px 60px;
        opacity:0.15;
        animation:gridmove 10s linear infinite;
        z-index:-1;
        }

        body.light::before{
        background-image:
        linear-gradient(rgba(0,102,204,0.08) 1px, transparent 1px),
        linear-gradient(90deg, rgba(0,102,204,0.08) 1px, transparent 1px);
        }

        @keyframes gridmove{
        0%{transform:translateY(0);}
        100%{transform:translateY(60px);}
        }

        /* RESPONSIVE */

        @media(max-width:1000px){
        .dashboard{grid-template-columns:repeat(2,1fr);}
        }

        @media(max-width:700px){
        .dashboard{grid-template-columns:1fr;}
        header{flex-direction:column;gap:15px;padding:20px;}
        }

        .dashboard .card:nth-child(7){
        grid-column:1 / 4;
        }

    </style>
    </head>

    <body>

        <header>

        <h1>AI Coding Intelligence Assessment Dashboard</h1>

          <div class="nav">
```

```
      <button onclick="location.href='student.html'">Students</button>
      <button onclick="location.href='codingSession.html'">Coding</button>
      <button onclick="location.href='reports.html'">Reports</button>
      <button onclick="openSettings()">Settings</button>
      <button onclick="location.href='../index.html'">Logout</button>

    </div>

</header>

    <div class="dashboard">
      <div class="card">
      <h3>REGISTERED STUDENTS</h3>
      <div class="big-number" id="students">0</div>
    </div>

<div class="card">
    <h3>AI PERFORMANCE STATUS</h3>
    <div class="big-number" id="predictionRate">0%</div>
    <p style="font-size:12px;">Overall coding performance prediction</p>
</div>

<div class="card">
    <h3>CODING SESSIONS</h3>
    <div class="big-number" id="sessions">0</div>
    <p style="font-size:12px;">Total coding submissions detected</p>
</div>

<div class="card">
<h3>AI CONTROL CENTER</h3>
    <div class="action-buttons">
      <button onclick="location.href='trainModel.html'">Train Model</button>
      <button onclick="location.href='reports.html'">Generate Report</button>
      <button onclick="location.href='alogin.html'">Admin</button>
    </div>
</div>

<div class="card">
    <h3>LIVE SYSTEM ACTIVITY</h3>
    <div class="feed" id="feed"></div>
</div>

<div class="card">
    <h3>PERFORMANCE ANALYTICS</h3>
    <canvas id="performanceChart"></canvas>
</div>

<div class="card">
    <h3>MOST PROGRAMMING LANGUAGES USED</h3>
    <canvas id="languageChart"></canvas>
```

```
    </div>

    </div>

    <script>

    /* FIX 2: wait for DOM before running (prevents slow render blocking) */
    window.addEventListener("DOMContentLoaded", () => {

    /* LOAD THEME */
    let savedTheme = localStorage.getItem("theme");
    if(savedTheme === "light"){
    document.body.classList.add("light");
    }

    /* SETTINGS */
    function openSettings(){
    window.location.href = "settings.html";
    }

    /* LOAD DATA */
    let students = JSON.parse(localStorage.getItem("students")) || [];
    let codingSessions = JSON.parse(localStorage.getItem("codingSessions")) ||
[];

    document.getElementById("students").innerText = students.length;
    document.getElementById("sessions").innerText = codingSessions.length;

    /* AI ANALYSIS */
    let improve = 0;
    let average = 0;
    let good = 0;
    let excellent = 0;

    codingSessions.forEach(session=>{
    if(session.prediction === "Excellent") excellent++;
    else if(session.prediction === "Good") good++;
    else if(session.prediction === "Average") average++;
    else if(session.prediction === "Needs Improvement") improve++;
    });

    /* PERFORMANCE RATE */
    let totalGoodPerformance = excellent + good;

    let performanceRate = codingSessions.length > 0
    ? Math.floor((totalGoodPerformance / codingSessions.length) * 100)
    : 0;

    document.getElementById("predictionRate").innerText = performanceRate +
"%";
```

```
/* LIVE FEED */
const feed = document.getElementById("feed");

codingSessions.reverse().slice(0,6).forEach(session=>{
let div = document.createElement("div");
div.innerText = session.student + " submitted " + session.lines + " lines |
Prediction: " + session.prediction;
feed.appendChild(div);
});

setInterval(()=>{
let messages = [
"AI analyzing student code...",
"Machine learning updating model...",
"Code complexity analysis running...",
"AI prediction system scanning sessions...",
"Student coding session synchronized..."
];

let msg = document.createElement("div");
msg.innerText = messages[Math.floor(Math.random()*messages.length)];

feed.prepend(msg);

if(feed.children.length > 8){
feed.removeChild(feed.lastChild);
}

},3000);

/* PERFORMANCE CHART */
const performanceCtx = document.getElementById("performanceChart");

new Chart(performanceCtx,{
type:"line",
data:{
labels:["Excellent","Good","Average","Needs Improvement"],
datasets:[{
label:"Performance",
data:[excellent,good,average,improve],
backgroundColor:"rgba(0,234,255,0.2)",
borderColor:"#00eaff",
pointRadius:5,
fill:true,
tension:0.4
}]
}
});
```

```
/* LANGUAGE CHART */
function getLanguageStats(){

let stats = {
HTML:0,
CSS:0,
Python:0,
JavaScript:0,
"C#":0,
"C++":0,
PHP:0,
Java:0
};

codingSessions.forEach(s=>{
if(stats[s.language] !== undefined){
stats[s.language]++;
}
});

return stats;
}

const languageCtx = document.getElementById("languageChart");

let languageChart;

function renderLanguageChart(){

let lang = getLanguageStats();

if(languageChart) languageChart.destroy();

languageChart = new Chart(languageCtx,{
type:"bar",
data:{
labels:["HTML","CSS","Python","JavaScript","C#","C++","PHP","Java"],
datasets:[{
label:"Usage",
data:[
lang.HTML,
lang.CSS,
lang.Python,
lang.JavaScript,
lang["C#"],
lang["C++"],
lang.PHP,
lang.Java
],
backgroundColor:"#00eaff",
```

```
        }]
        }
        });

        }

        renderLanguageChart();

        setInterval(()=>{
        codingSessions = JSON.parse(localStorage.getItem("codingSessions")) || [];
        renderLanguageChart();
        },3000);

        });

        function openSettings(){

        window.location.href =
        "settings.html";

        }

        </script>

    </body>
    </html>
```

APPENDIX B. USER'S GUIDE

USER'S GUIDE

AN INTELLIGENT MACHINE LEARNING SYSTEM FOR PROGRAMMING DIFFICULTY ASSESSMENT AND CODING ENGAGEMENT ENHANCEMENT

1. Introduction

This User's Guide provides step-by-step instructions for using the **An Intelligent Machine Learning System for Programming Difficulty Assessment and Coding Engagement Enhancement**. The system is developed to assist in managing student information, assessing programming code, identifying programming languages, detecting coding errors or correctness, generating recommendations, and viewing assessment reports.

The system consists of three major features:

1) **Student Management**
2) **Coding Assessment**
3) **Report**

This guide is intended for authorized users, instructors, administrators, and students who will interact with the system.

2. System Access

To access the system, the user must have a compatible device and a supported web browser.

Step 1: Open the Browser

Open a web browser such as Google Chrome, Mozilla Firefox, or Microsoft Edge.

Step 2: Access the System

Enter the system's assigned web address in the browser's address bar.

Step 3: Login

Enter the required account credentials:

- Username
- Password

Click the **Login** button.

If the credentials are valid, the system will display the user's dashboard.

3. Dashboard

After successful login, the system displays the main dashboard.

The dashboard provides access to the major functions of the system:

- Student Management
- Coding Assessment
- Report
- Logout

The dashboard serves as the main navigation area of the system.

4. Student Management

The **Student Management** feature allows authorized users to manage student information.

Users can:

- Add student records
- View student records
- Search student records
- Edit student information
- Delete student records

4.1 Adding a student

To add a new student:

1. Click **Student Management**.
2. Click **Add Student**.
3. Enter the required student information.
4. Review the information.
5. Click **Save**.
6. The system stores the student record.
7. The new student will appear in the student list.

The information may include:

- Student ID
- Full Name

- Gender
- Year Level
- Program/Course
- Contact Information
- Other required information

4.2 Viewing Student Information

To view a student's information:

1. Open **Student Management**.
2. Locate the desired student.
3. Click **View**.
4. The student's information will be displayed.

4.3 Searching for a Student

To search for a student:

1. Open **Student Management**.
2. Locate the search field.
3. Enter the student's name or ID number.
4. Click **Search**, if required.
5. The system displays matching student records.

4.4 Editing Student Information

To update a student record:

1. Locate the student.

2. Click **Edit**.
3. Modify the required information.
4. Review the changes.
5. Click **Update** or **Save**.
6. The system updates the student record.

4.5 Deleting a student

To delete a student record:

1. Locate the student record.
2. Click **Delete**.
3. Review the selected record.
4. Confirm the deletion.
5. The system removes the selected record.

Reminder: Users should verify the selected student before confirming deletion.

5. Coding Assessment

The **Coding Assessment** is the main intelligent feature of the system.

It allows the user to submit source code and have the system analyze it.

The system is designed to:

1. Identify the programming language used.
2. Analyze the submitted code.
3. Determine whether the code is correct or contains errors.
4. Identify possible coding difficulties.

5. Generate recommendations.
6. Record the assessment result.

6. Starting a Coding Assessment

To perform a coding assessment:

1. Open **Coding Assessment**.
2. Select the student to be assessed.
3. Enter or paste the source code into the coding area.
4. Review the submitted code.
5. Click **Analyze Code** or the corresponding assessment button.
6. Wait while the system processes the code.
7. Review the assessment results.

7. Programming Language Detection

The system analyzes the submitted source code to determine the programming language being used.

Depending on the programming languages supported by the implemented system, the result may identify languages such as:

- HTML
- CSS
- JavaScript
- Python
- Java
- C++

- Other supported languages

Procedure

1. Enter the source code.
2. Click **Analyze Code**.
3. Wait for the analysis.
4. Check the **Detected Programming Language** result.

The detected language is based on recognizable characteristics or syntax within the submitted code.

8. Code Correctness and Error Detection

After identifying the programming language, the system analyzes the submitted code.

The system may classify the submitted code as:

- **Correct**
- **Contains Error**
- **Needs Improvement**

Procedure

1. Submit the source code.
2. Start the assessment.
3. Wait for the analysis to finish.
4. Review the code status.
5. Examine the detected error or problem, if available.

6. Read the recommendation generated by the system.

The purpose of this feature is to provide useful feedback that can help students understand and improve their coding performance.

9. Viewing the Recommendation

After analyzing the source code, the system generates a recommendation based on the assessment.

The recommendation may guide the student to:

- Review programming syntax.
- Check the identified error.
- Review a particular programming concept.
- Correct a missing or incorrect element.
- Improve the programming logic.
- Practice the related programming topic.
- Resubmit the corrected code.

Procedure

1. Complete the coding assessment.
2. Locate the **Recommendation** section.
3. Read the generated recommendation.
4. Apply the suggested correction.
5. Modify the source code.
6. Submit the code again for reassessment.

10. Reassessment

The system allows the student to improve the submitted code and perform another assessment.

Procedure

1. Review the previous assessment.
2. Identify the detected problem.
3. Follow the generated recommendation.
4. Correct the source code.
5. Submit the revised code.
6. Run the assessment again.
7. Review the new result.

This process allows students to practice coding and learn from identified errors.

11. Report

The **Report** feature provides organized information about coding assessments.

It allows authorized users to review assessment records and monitor students' coding performance.

The report may contain:

- Student Name
- Student ID
- Assessment Date
- Programming Language

- Submitted Code
- Code Status
- Detected Error
- Recommendation
- Assessment Result

12. Viewing Reports

To view an assessment report:

1. Click **Report**.
2. Select the desired student or assessment record.
3. Apply available filters if necessary.
4. Click **View**.
5. Review the displayed assessment information.

13. Searching Reports

To search for a particular assessment record:

1. Open the **Report** module.
2. Locate the search field.
3. Enter the student's name or ID.
4. Click **Search**.
5. Review the matching records.

If available, reports may also be filtered according to:

- Student

- Year Level
- Programming Language
- Assessment Date
- Assessment Status

14. Monitoring Coding Performance

The Report module can be used by authorized users to review students' coding assessment records.

The instructor or administrator can review:

1. Completed assessments.
2. Programming languages detected.
3. Correct code submissions.
4. Code submissions containing errors.
5. Identified coding difficulties.
6. Generated recommendations.
7. Assessment history.

These records can provide useful information for monitoring students' programming activities and identifying areas that may require additional instruction or practice.

15. Logout

After completing the activities, the user should log out of the system.

Procedure

1. Locate the **Logout** button.
2. Click **Logout**.
3. The system ends the current session.
4. The user is returned to the login page or system landing page.

Users should always log out when using a shared computer.

16. Troubleshooting

Problem	Possible Cause	Solution
Cannot log in	Incorrect credentials	Check the username and password
Student cannot be found	Incorrect search information	Verify the student's name or ID
Student information is not saved	Required field is missing	Complete all required fields
Code cannot be analyzed	Empty or unsupported code	Submit valid supported source code
Wrong programming language detected	Code has insufficient recognizable syntax	Submit a more complete code sample
Error is detected	Submitted code contains an error	Review the identified problem
Recommendation is unclear	The coding problem may require further explanation	Consult the instructor and review the relevant programming concept
No report appears	No assessment record exists	Complete a coding assessment first
System does not load	Browser or internet problem	Refresh the page or check the connection
System responds slowly	Network or server issue	Check the internet connection and try again

Table 3. Troubleshoot

17. Important Reminders

Users should follow these guidelines when using the system:

1. Enter accurate student information.
2. Keep account credentials confidential.
3. Submit valid source code for assessment.
4. Carefully review the detected programming language.
5. Read the generated recommendation.
6. Correct identified coding errors when possible.
7. Use the system's recommendations as learning assistance.
8. Consult the instructor when an assessment result requires further explanation.
9. Verify important assessment information before using it for academic decisions.
10. Log out after completing the session.

18. System Workflow

The overall system workflow is:

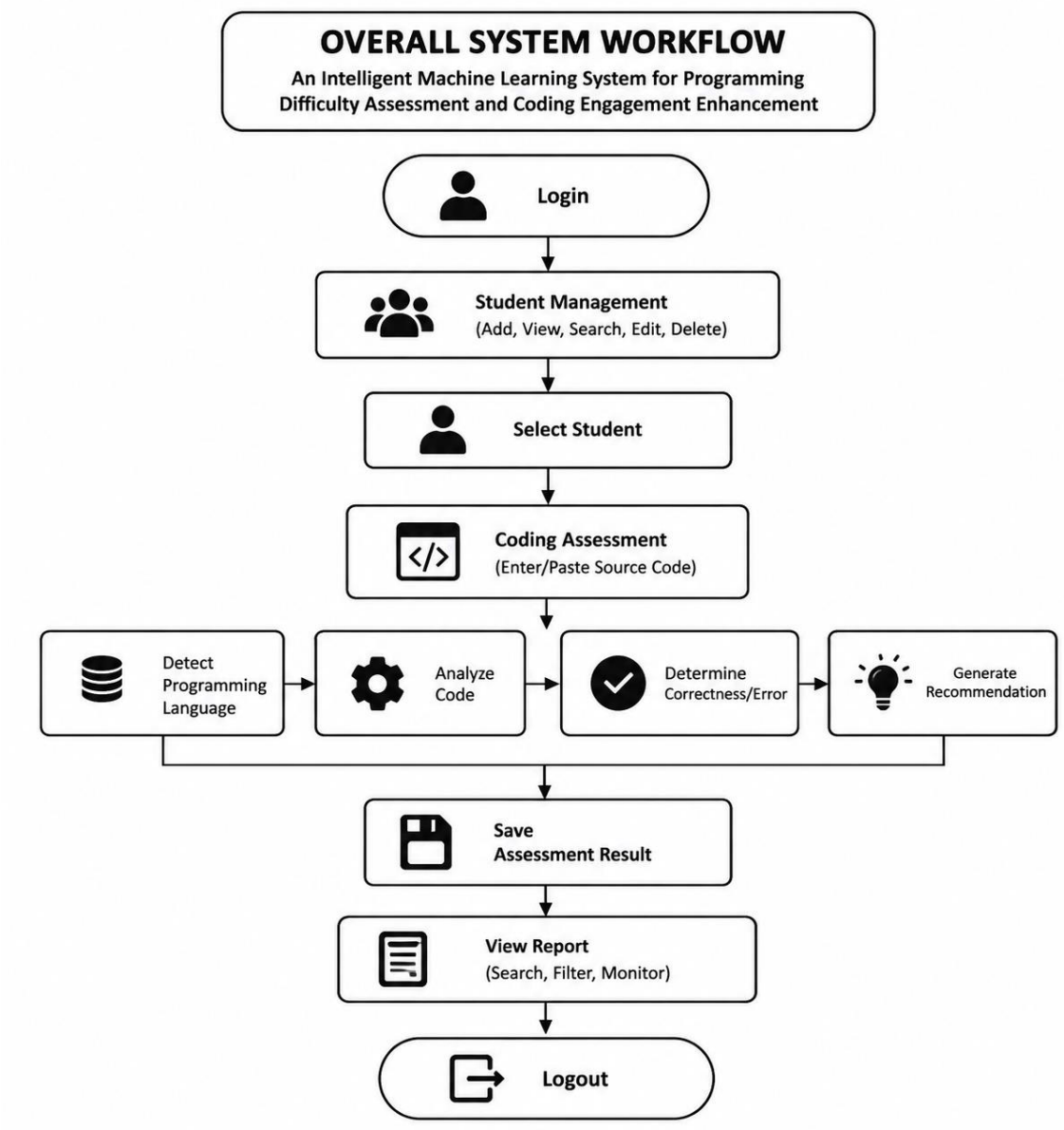


Figure 12. The Overall System Workflow

19. Conclusion

The **An Intelligent Machine Learning System for Programming Difficulty Assessment and Coding Engagement Enhancement** provides users with an

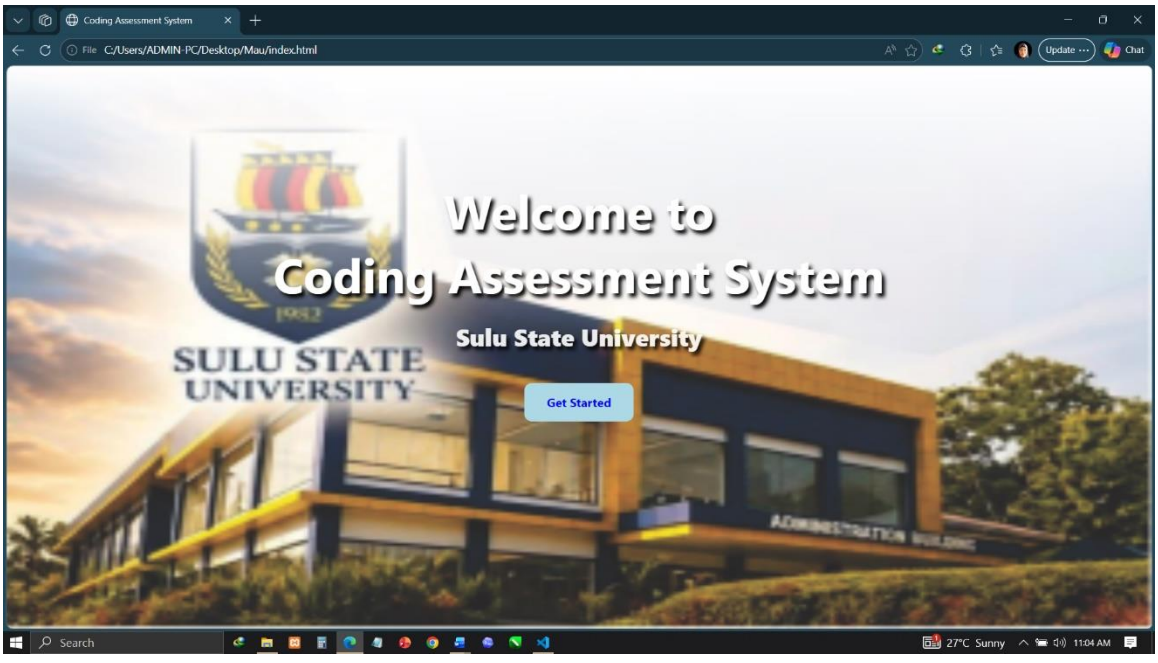
organized platform for student management, automated coding assessment, intelligent recommendation, and assessment reporting.

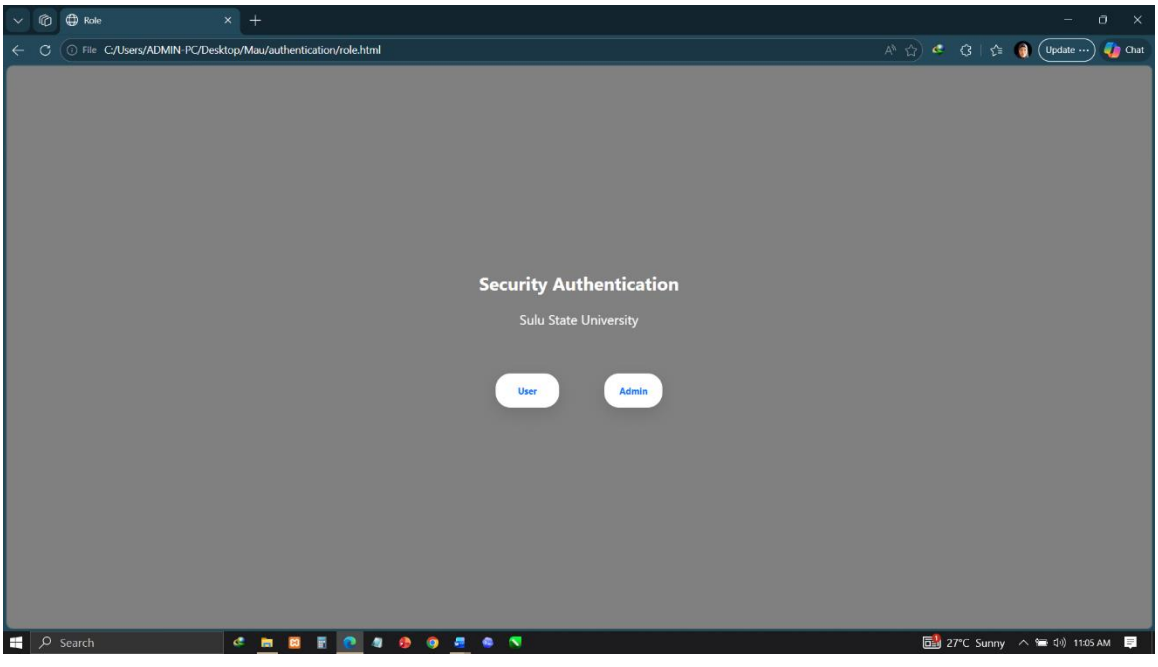
Through the **Student Management** feature, authorized users can maintain student records. Through **Coding Assessment**, submitted code can be analyzed to identify the programming language, determine code correctness or errors, and provide recommendations. Through the **Report** feature, users can review and monitor assessment information.

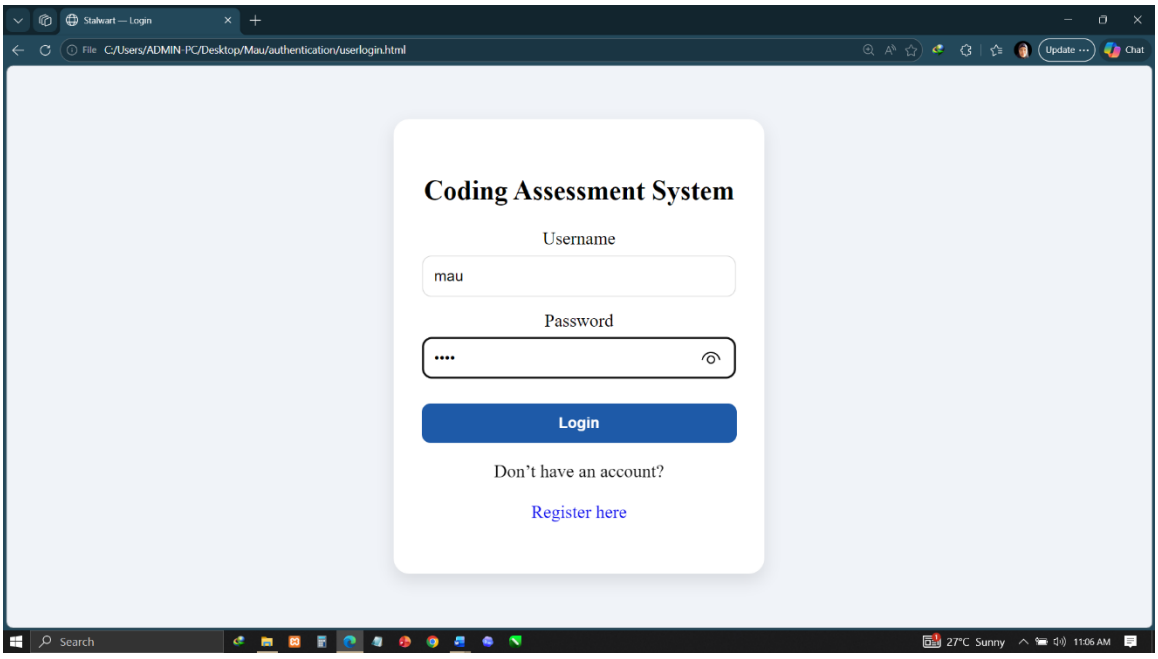
The system is intended to function as an educational support tool. Its automated results and recommendations should be used together with proper instruction, programming practice, and instructor guidance.

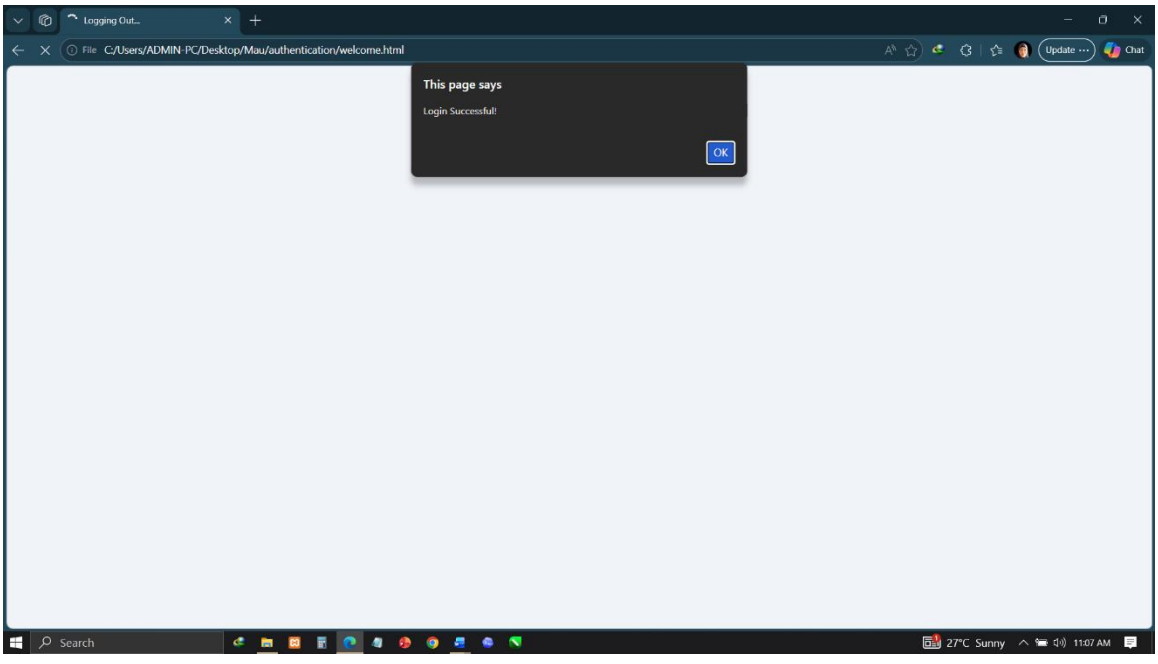
APPENDIX C. SCREEN LAYOUTS

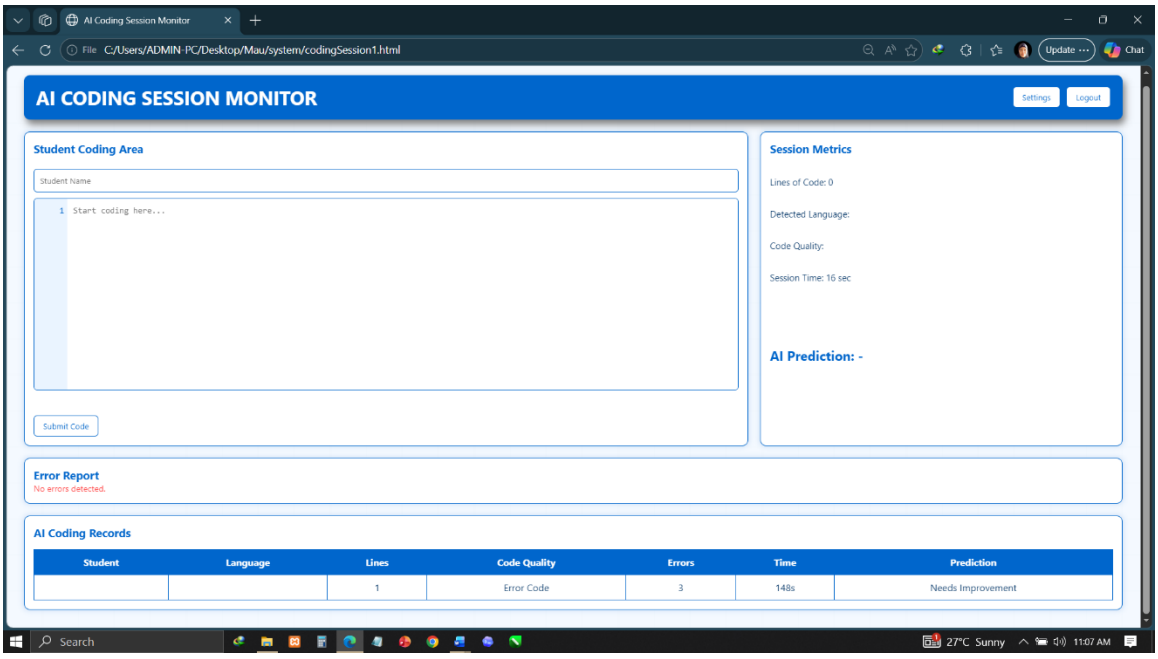
User Start



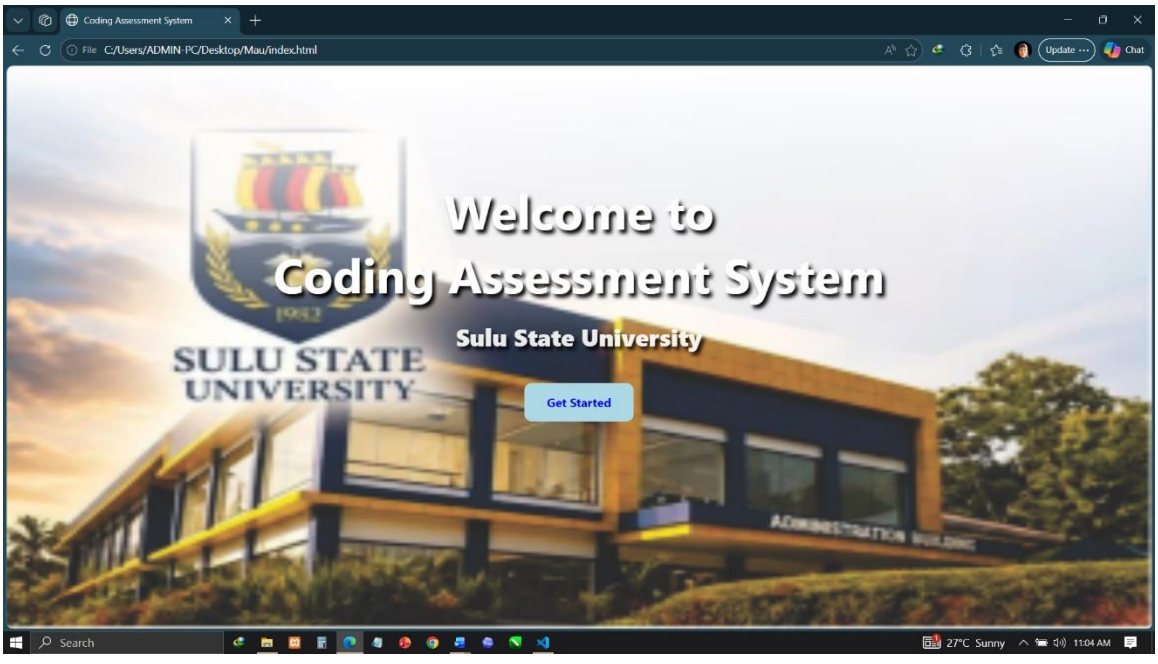


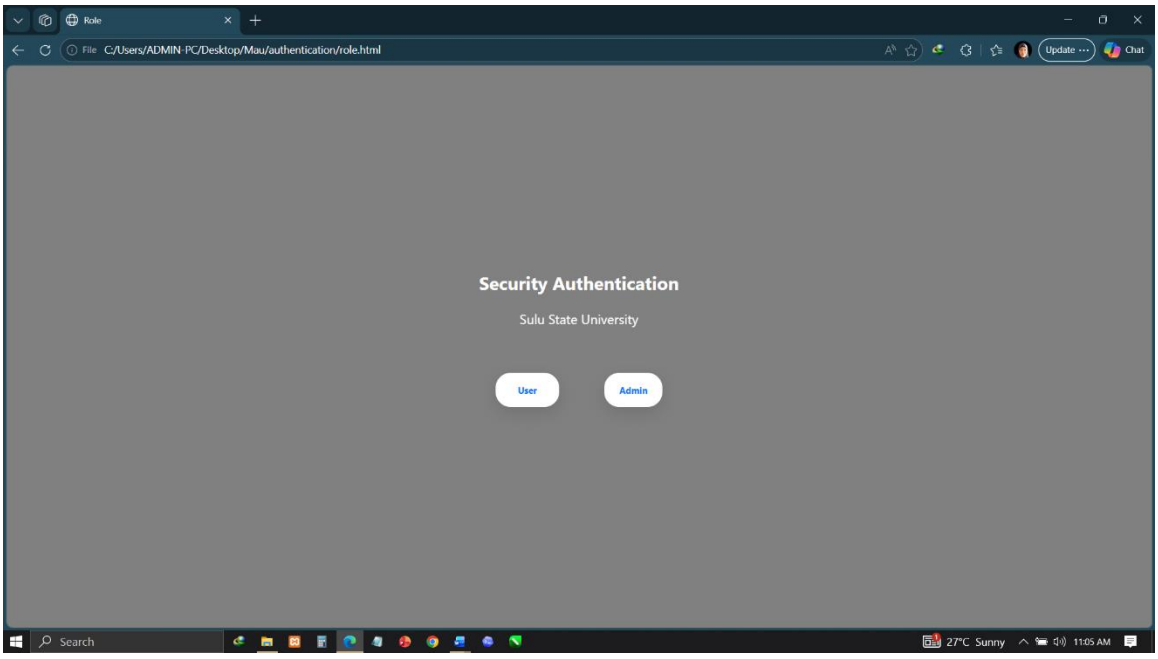

Stalwart — Login
File C:/Users/ADMIN-PC/Desktop/Mau/authentication/userlogin.html
Update
Chat
Coding Assessment System
Username
mau
Password
Login
Don't have an account?
Register here
Search
27°C Sunny
11:06 AM

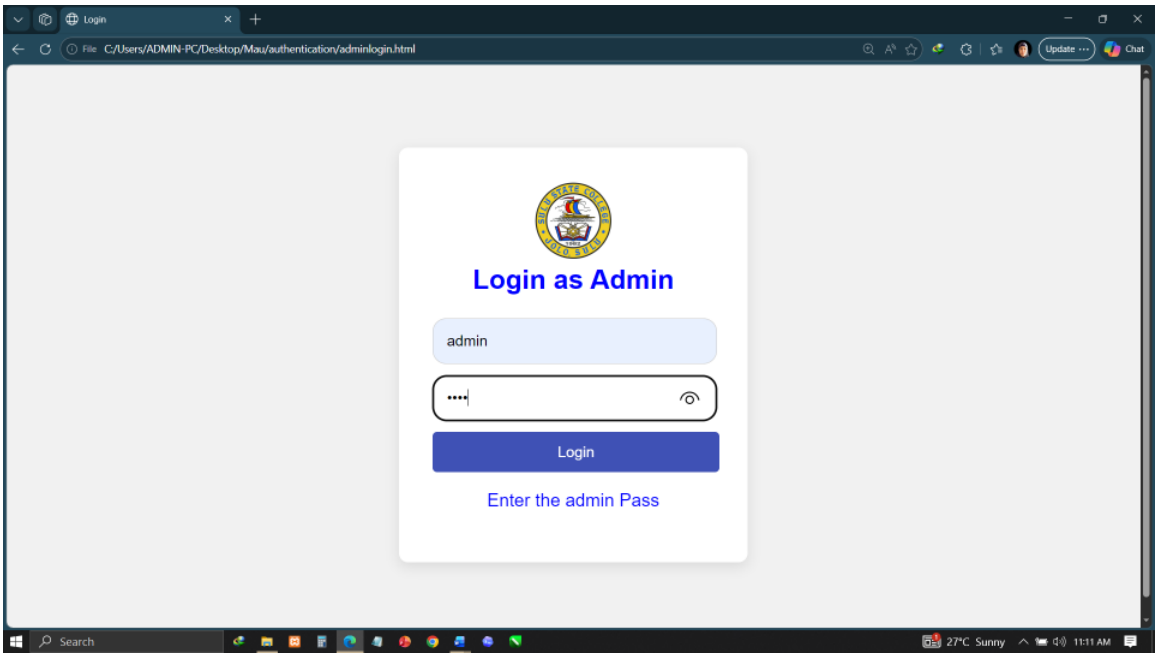

Logging Out...
File C:/Users/ADMIN-PC/Desktop/Mau/authentication/welcome.html
Update
Chat
This page says
Login Successful!
OK
Search
27°C Sunny
11:07 AM

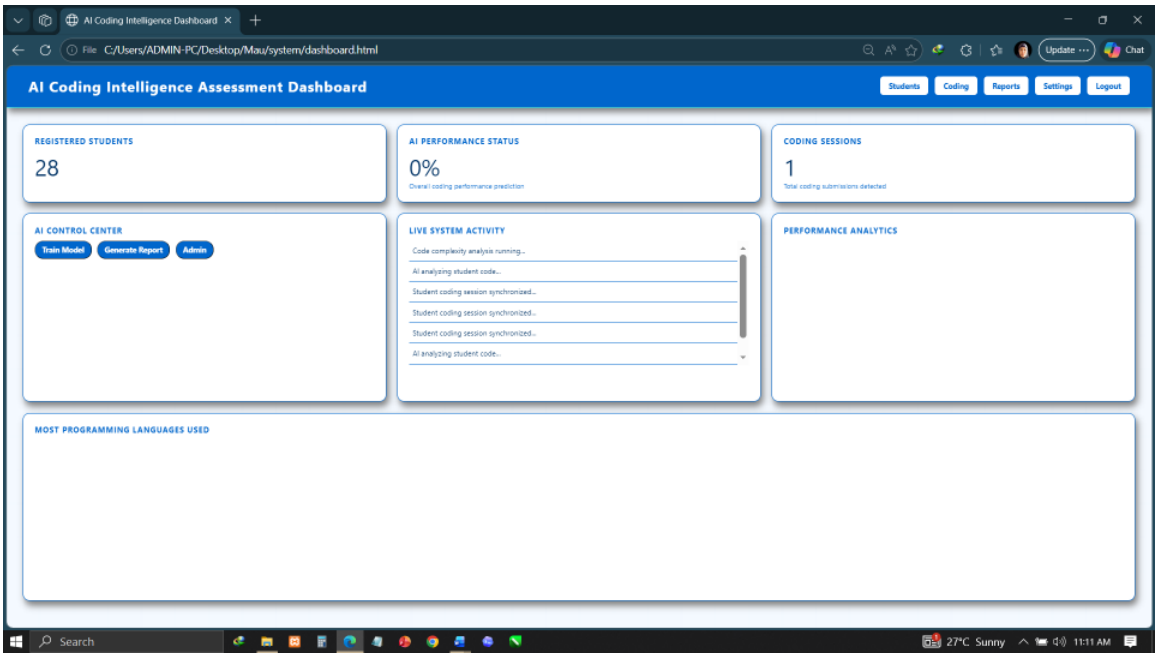

AI Coding Session Monitor
File C:/Users/ADMIN-PC/Desktop/Mau/system/codingSession1.html
Update
Chat
AI CODING SESSION MONITOR
Settings
Logout
Student Coding Area
Student Name
1 Start coding here...
Submit Code
Session Metrics
Lines of Code: 0
Detected Language:
Code Quality:
Session Time: 16 sec
AI Prediction: -
Error Report
No errors detected.
AI Coding Records
Student
Language
Lines
Code Quality
Errors
Time
Prediction
1
Error Code
3
148s
Needs Improvement
Search
27°C Sunny
11:07 AM

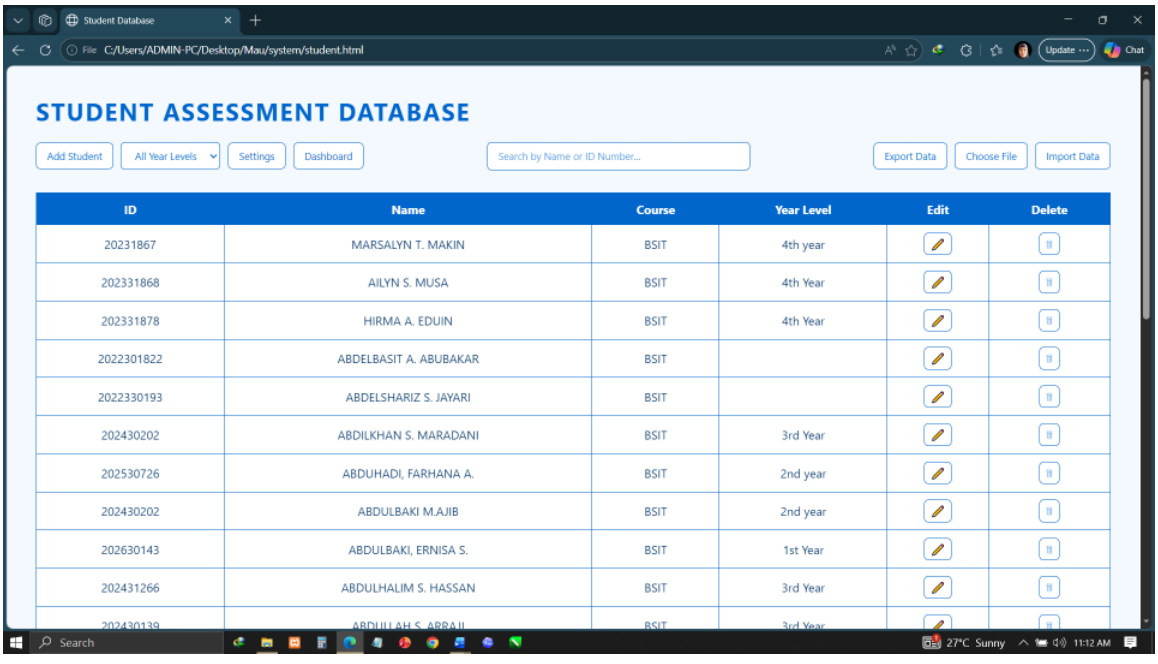
Admin Start

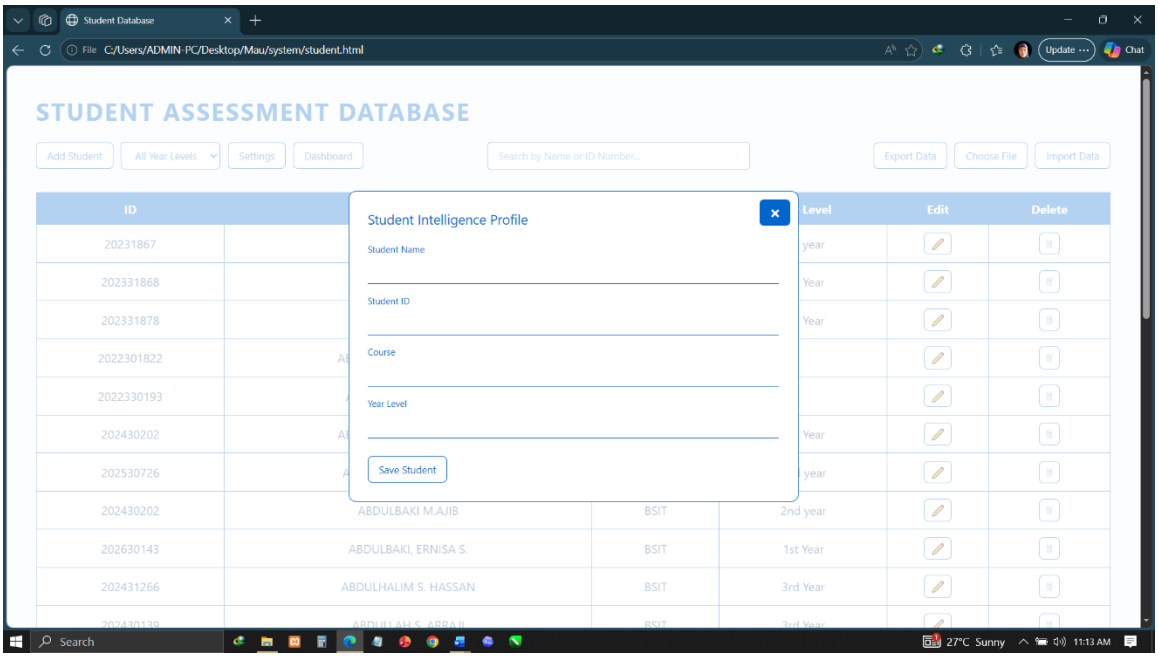

Coding Assessment System
File C:/Users/ADMIN-PC/Desktop/Mau/index.html
Update
Chat
Welcome to
Coding Assessment System
Sulu State University
SULU STATE UNIVERSITY
Get Started
Search
27°C Sunny
11:04 AM

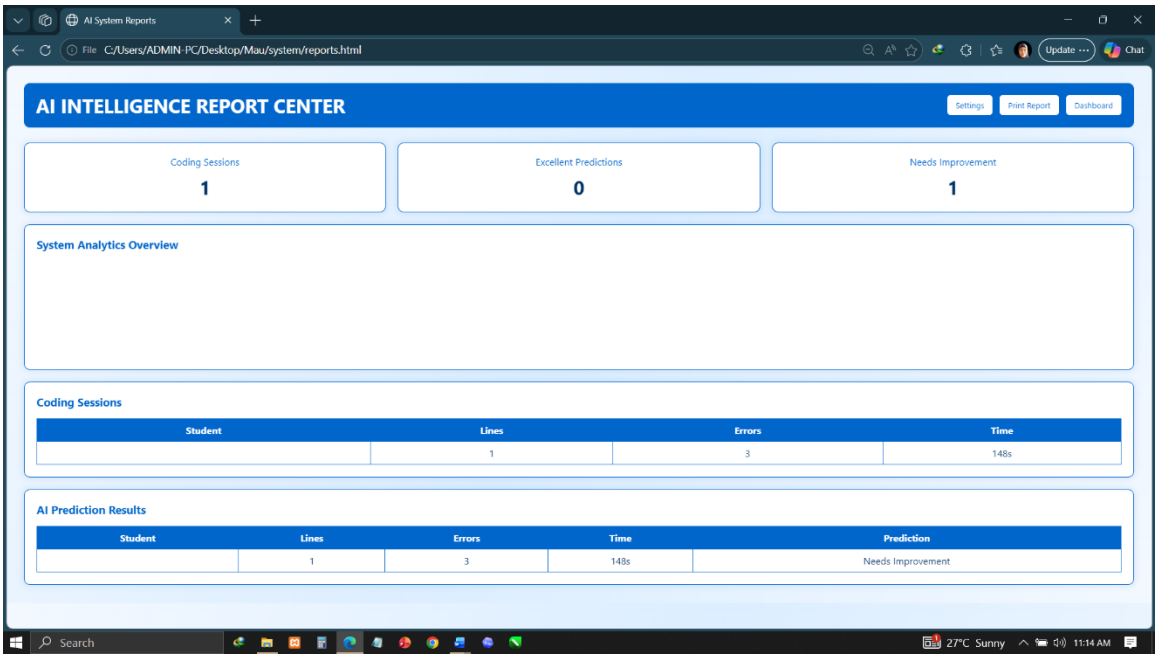

Role
File C:/Users/ADMIN-PC/Desktop/Mau/authentication/role.html
Update
Chat
Security Authentication
Sulu State University
User
Admin
Search
27°C Sunny
11:05 AM


Login
File C:/Users/ADMIN-PC/Desktop/Mau/authentication/adminlogin.html
Update
Chat
Login as Admin
admin
Login
Enter the admin Pass
Search
27°C Sunny
11:11 AM


AI Coding Intelligence Dashboard
File C:/Users/ADMIN-PC/Desktop/Mau/system/dashboard.html
Update
Chat
AI Coding Intelligence Assessment Dashboard
Students
Coding
Reports
Settings
Logout
REGISTERED STUDENTS
28
AI PERFORMANCE STATUS
0%
Overall coding performance prediction
CODING SESSIONS
1
Total coding submissions detected
AI CONTROL CENTER
Train Model
Generate Report
Admin
LIVE SYSTEM ACTIVITY
Code complexity analysis running...
AI analyzing student code...
Student coding session synchronized...
Student coding session synchronized...
Student coding session synchronized...
AI analyzing student code...
PERFORMANCE ANALYTICS
MOST PROGRAMMING LANGUAGES USED
Search
27°C Sunny
11:11 AM


Student Database
File C:/Users/ADMIN-PC/Desktop/Mau/system/student.html
Update
Chat
STUDENT ASSESSMENT DATABASE
Add Student
All Year Levels
Settings
Dashboard
Search by Name or ID Number...
Export Data
Choose File
Import Data
ID
Name
Course
Year Level
Edit
Delete
20231867 MARSALYN T. MAKIN BSIT 4th year
202331868 AILYN S. MUSA BSIT 4th Year
202331878 HIRMA A. EDUIN BSIT 4th Year
2022301822 ABDELBASIT A. ABUBAKAR BSIT
2022330193 ABDELSHARIZ S. JAYARI BSIT
202430202 ABDILKHAN S. MARADANI BSIT 3rd Year
202530726 ABDUHADI, FARHANA A. BSIT 2nd year
202430202 ABDULBAKI M.AJIB BSIT 2nd year
202630143 ABDULBAKI, ERNISA S. BSIT 1st Year
202431266 ABDULHALIM S. HASSAN BSIT 3rd Year
Search
27°C Sunny
11:12 AM


Student Database
File C:/Users/ADMIN-PC/Desktop/Mau/system/student.html
Update
Chat
STUDENT ASSESSMENT DATABASE
Add Student
All Year Levels
Settings
Dashboard
Search by Name or ID Number...
Export Data
Choose File
Import Data
ID
Level
Edit
Delete
Student Intelligence Profile
Student Name
Student ID
Course
Year Level
Save Student
20231867
202331868
202331878
2022301822
2022330193
202430202
202530726
202430202
ABDULBAKI M.AJIB
BSIT
2nd year
202630143
ABDULBAKI, ERNISA S.
BSIT
1st Year
202431266
ABDULHALIM S. HASSAN
BSIT
3rd Year
Search
27°C Sunny
11:13 AM


AI System Reports
File C:/Users/ADMIN-PC/Desktop/Mau/system/reports.html
Update
Chat
AI INTELLIGENCE REPORT CENTER
Settings
Print Report
Dashboard
Coding Sessions
1
Excellent Predictions
0
Needs Improvement
1
System Analytics Overview
Coding Sessions
Student
Lines
Errors
Time
1
3
148s
AI Prediction Results
Student
Lines
Errors
Time
Prediction
1
3
148s
Needs Improvement
Search
27°C Sunny
11:14 AM

APPENDIX D. TEST RESULTS

1. Functional Test Results

The functional testing was conducted to determine whether the developed system performed its intended functions correctly. The testing focused on the three major features of the system: **Student Management, Coding Assessment, and Report**. Each function was tested using different test cases to verify whether the expected output was produced. The results showed that the system was able to perform the specified functions according to their intended requirements.

Table 4. Student Management Test Results

Test Case	Expected Result	Actual Result	Status
Add Student	Student information is successfully saved	Student information was saved successfully	Passed
View Student	Selected student information is displayed	Student information was displayed	Passed
Search Student	Matching student record is displayed	Matching record was displayed	Passed
Edit Student	Updated information is saved	Information was successfully updated	Passed
Delete Student	Selected student record is removed	Record was successfully removed	Passed

Interpretation:

The Student Management feature successfully performed the tested functions. The system was able to add, display, search, update, and delete student records. This indicates that the student information management component was functioning according to the specified requirements.

2. Coding Assessment Test Results

The Coding Assessment feature was tested to determine whether the system could analyze submitted source code, identify the programming language, determine code correctness or errors, and generate recommendations.

Table 5. Coding Assessment Test Results

Test Case	Expected Result	Actual Result	Status
Submit valid source code	System accepts the submitted code	Code was successfully submitted	Passed
Detect programming language	System identifies the programming language	Programming language was identified	Passed
Analyze correct code	System identifies code as correct	Correct code was identified	Passed
Analyze code with error	System identifies the presence of an error	Error was detected	Passed
Generate recommendation	System provides an appropriate recommendation	Recommendation was generated	Passed
Resubmit corrected code	System analyzes the revised code	Revised code was successfully analyzed	Passed

Interpretation:

The Coding Assessment feature successfully processed the submitted source code. The system was able to distinguish the programming language, analyze the submitted code, identify whether the code was correct or contained an error, and provide recommendations based on the assessment. These functions support the primary purpose of the system, which is to assist students in identifying programming difficulties and improving their coding performance.

3. Report Test Results

The Report feature was tested to determine whether assessment information was properly recorded and presented to authorized users.

Table 6. Report Test Results

Test Case	Expected Result	Actual Result	Status
View assessment report	Assessment records are displayed	Assessment records were displayed	Passed
Search report	Matching assessment record is displayed	Matching record was displayed	Passed
View student assessment history	Student's assessment records are displayed	Assessment history was displayed	Passed
Display programming language	Detected language is shown	Programming language was displayed	Passed
Display code status	Correct/error status is shown	Code status was displayed	Passed
Display recommendation	Generated recommendation is shown	Recommendation was displayed	Passed

Interpretation:

The Report feature successfully displayed the required assessment information. It allowed users to review student coding assessment records, detected programming languages, code status, and generated recommendations. The results demonstrate that the reporting component can organize assessment information for easier monitoring and review.

4. Overall Functional Test Results

Table 7. Overall Functional Testing

System Feature	Functions Tested	Passed	Failed	Success Rate
Student Management	5	5	0	100%
Coding Assessment	6	6	0	100%
Report	6	6	0	100%
Total	**17**	**17**	**0**	**100%**

Interpretation:

Based on the sample functional test results, all 17 tested functions were successfully executed, resulting in a 100% functional test success rate. The Student Management, Coding Assessment, and Report features performed their intended functions without failed test cases during the testing process. The result indicates that the system's core functionalities were operational based on the defined test cases.

5. Coding Analysis Test Results

Additional testing was conducted specifically for the intelligent Coding Assessment component. Different source-code conditions were used to determine whether the system could distinguish programming languages and identify code correctness or errors.

Table 8. Coding Analysis Results

Test Case	Code Condition	Expected Output	System Output	Result
Test 1	Correct source code	Correct	Correct	Passed
Test 2	Source code containing syntax error	Error detected	Error detected	Passed

Test Case	Code Condition	Expected Output	System Output	Result
Test 3	Source code containing missing statement/symbol	Error detected	Error detected	Passed
Test 4	Source code with incorrect syntax	Error detected	Error detected	Passed
Test 5	Valid code from supported language	Language identified	Language identified	Passed
Test 6	Code requiring improvement	Recommendation generated	Recommendation generated	Passed

Interpretation:

The coding analysis tests demonstrate that the system was able to process different coding conditions and provide the expected assessment results. Correct source code was identified as correct, while source code containing detectable errors was identified accordingly. The system also generated recommendations intended to guide students toward correcting or improving their submitted code.

6. Recommendation Test Results

The recommendation component was tested to determine whether the system could provide feedback after identifying coding difficulties.

Table 9. Recommendation Testing

Test Case	Detected Condition	Expected Recommendation	Actual Result	Status
1	Syntax error	Review syntax	Recommendation generated	Passed
2	Missing element	Check missing element	Recommendation generated	Passed
3	Incorrect statement	Review statement	Recommendation generated	Passed
4	Correct code	Provide positive assessment/next learning guidance	Recommendation generated	Passed

Test Case	Detected Condition	Expected Recommendation	Actual Result	Status
5	Coding improvement required	Practice/review related concept	Recommendation generated	Passed

Interpretation:

The recommendation testing showed that the system was capable of producing feedback based on the identified coding condition. The recommendations are intended to provide students with additional guidance after an assessment and encourage them to review, correct, and improve their programming work.

7. Usability Test Results

The system was also evaluated in terms of usability to determine whether users could navigate and operate the major system functions.

Table 10. Usability Test Results

Criteria	Description	Result
Ease of Use	Users can easily understand how to use the system	Satisfactory
Navigation	Users can move between system modules	Satisfactory
Interface	System information and controls are understandable	Satisfactory
Student Management	Student records can be managed efficiently	Satisfactory
Coding Assessment	Users can submit and assess source code	Satisfactory
Reports	Assessment information can be viewed clearly	Satisfactory

Interpretation:

The usability testing indicates that the system provides an understandable interface for performing the major functions. Users were able to navigate through

the system, manage student information, perform coding assessments, and view reports.

8. Summary of Test Results

The testing results indicate that the developed system successfully performed its major functions based on the defined test cases. The **Student Management** feature successfully handled student records, while the **Coding Assessment** feature was able to process source code, identify the programming language, detect coding errors or correctness, and generate recommendations. The **Report** feature successfully presented assessment information for monitoring and review.

Overall, the test results demonstrate that the system's major components were functioning according to their intended requirements. The results also indicate that the system can serve as an educational support tool for programming assessment by providing automated analysis and recommendations. However, the accuracy of machine-learning-based code analysis and recommendations should continue to be evaluated using a larger and more diverse set of programming submissions to further determine the system's reliability and effectiveness.

APPENDIX E. PICTURES SHOWCASING DATA GATHERING





APPENDIX F. CURRICULUM VITAE

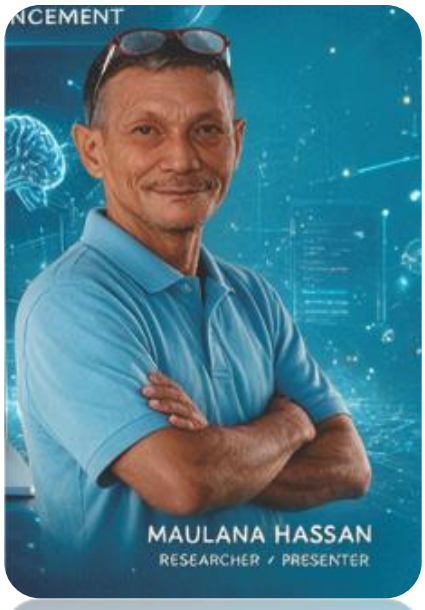
PERSONAL STATEMENT

We, the Information Technology student with a strong interest in programming, software development, machine learning, and emerging technologies. Throughout our academic journey, we have developed our technical knowledge and problem-solving skills through various projects, system development activities, and research, including the development of an intelligent machine learning system for programming difficulty assessment and coding engagement enhancement.

We are a dedicated, responsible, and motivated individual who is willing to learn, adapt to new challenges, and continuously improve my skills. We aspire to gain practical experience, contribute my knowledge and abilities to an organization, and become a competent IT professional who can create meaningful and innovative technology solutions.

We have sincerely dedicated our time, effort, and perseverance to the completion of this research. Every part of this study reflects our commitment, cooperation, and determination to accomplish this work to the best of our abilities. Attached to this research is our Biodata, which provides a brief background about the researchers and serves as a testament to the people behind this study.

PERSONAL DATA



Name:	Maulana K. Hassan
Gender:	Male
Age:	53
Date of Birth:	Nov, 27 1972
Place of Birth:	Biid, Parang Sulu
Address:	Suh, Panamao Sulu
Email:	mcmauladg@gmail.com
Contact:	09356725752

EDUCATIONAL ATTAINMENT

College:	Sulu State University BSIT 4th Year – A, to present
	Notre Dame of Jolo College BNS - 1992-1994, Quitted
Secondary:	Soutwestern Mindanao Islamic Institute 1987-1992, Graduate
Primary:	Baliwasan Central School 1978-1983, Graduate

CAREER OBJECTIVES

To obtain an opportunity where I can apply and further develop my knowledge and skills in Information Technology, particularly in programming, software development, machine learning, database management, and system development. Through my academic experience and thesis project, I aim to contribute my technical abilities, problem-solving skills, creativity, and willingness to learn in developing effective and innovative technology solutions. I am committed to continuously improving my skills, gaining practical industry experience, working effectively with others, and contributing positively to the goals and success of the organization while building a meaningful career in the field of Information Technology.

PERSONAL DATA



Name:	Hirma A. Eduin
Gender:	Female
Age:	30
Date of Birth:	Sept. 27 1996
Place of Birth:	Bunasi, Panglima Estino
Address:	Datag, Panamao Sulu
Email:	hirmaeduin1@gmail.com
Contact:	09100389474

EDUCATIONAL ATTAINMENT

College:	Sulu State University BSIT 4th Year – A, to present
Secondary:	Patibulan National High School 2023 - Graduate
Primary:	Baliwasan Central School 2012 - Graduate

CAREER OBJECTIVES

To obtain an opportunity where I can apply and further develop my knowledge and skills in Information Technology, particularly in programming, software development, machine learning, database management, and system development. Through my academic experience and thesis project, I aim to contribute my technical abilities, problem-solving skills, creativity, and willingness to learn in developing effective and innovative technology solutions. I am committed to continuously improving my skills, gaining practical industry experience, working effectively with others, and contributing positively to the goals and success of the organization while building a meaningful career in the field of Information Technology.